



Université de Tunis El Manar – Faculté des Sciences de Tunis

Département Informatique

En partenariat avec Tunisie Telecom

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de
Licence Nationale en Informatique

Sécurisation des flux de communication pour un écosystème IoT simulé

Architecture E2E – PKI – TLS/mTLS – SIEM – ML

Réalisé par :

Amine GHANNOUCHI

Encadrante universitaire : Mme. **ELLOUZE NOURHENE** – FST

Encadrant entreprise : M. Moez **KHLIFI** – Tunisie Telecom

Année universitaire **2025 – 2026**

Dédicace

*À mes parents, pour leur soutien indéfectible
et leur confiance tout au long de ce parcours.*

*À mes enseignants de la Faculté des Sciences de Tunis
qui m'ont transmis la passion de l'informatique.*

*À tous ceux qui œuvrent pour la sécurité
des systèmes d'information de demain.*

Remerciements

Ce travail n'aurait pu aboutir sans le concours de nombreuses personnes à qui je tiens à exprimer ma gratitude.

Je remercie tout d'abord **Mme. ELLOUZE NOURHENE**, encadrante universitaire à la Faculté des Sciences de Tunis, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de ce projet.

Mes remerciements vont également à **M. Moez KHLIFI** de *Tunisie Telecom* pour m'avoir accueilli au sein de son équipe, orienté vers des problématiques réelles de l'industrie télécom et soutenu dans mes expérimentations techniques.

Je remercie l'ensemble du corps enseignant du **Département Informatique de la FST** pour la qualité de la formation dispensée.

Un grand merci aux équipes des projets open-source **Wazuh**, **HashiCorp Vault**, **Mosquitto** et **Suricata** dont les documentations exhaustives ont grandement facilité l'implémentation.

Enfin, je remercie ma famille et mes amis pour leur encouragement constant durant cette année académique.

Résumé

Mots-clés : IoT, sécurité, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, SIEM, IsolationForest, RandomForest, détection d'anomalies, GNS3, Scrum.

L'essor de l'Internet des Objets (IoT) dans les environnements des opérateurs de télécommunications engendre des flux de données massifs et hétérogènes exposés à des risques croissants d'interception, d'altération et de compromission. Ce projet de fin d'études, réalisé en partenariat avec *Tunisie Telecom*, aborde la problématique de la **sécurisation de bout en bout des flux de communication d'un écosystème IoT simulé**.

La démarche adoptée suit la méthodologie **Scrum** et s'appuie sur un laboratoire simulé combinant **GNS3** pour la topologie réseau et **Docker** pour les nœuds applicatifs. Nous avons conçu et déployé une architecture composée de six éléments majeurs : (1) une **infrastructure à clés publiques (PKI)** basée sur HashiCorp Vault avec une hiérarchie CA racine – CA intermédiaire ; (2) un **broker MQTT Mosquitto** configuré en TLS 1.3 et mTLS obligatoire ; (3) un **simulateur de flotte IoT** jouant 76 000 événements réels issus d'un dataset CSV couvrant sept types d'attaques ; (4) une **analyse réseau** par Suricata avec règles personnalisées ; (5) un **SIEM Wazuh** ingérant les alertes et corrélant les événements ; (6) un **pipeline de détection d'anomalies** par Machine Learning (IsolationForest + RandomForest).

Les résultats obtenus démontrent l'efficacité de l'approche : le RandomForest atteint un F1-score de **0,994** et un ROC-AUC de **0,999** pour la détection binaire, tandis que l'IsolationForest, en mode non supervisé, obtient **0,930 / 0,959**. Les tests de performance montrent que l'overhead TLS 1.3/mTLS sur le RTT MQTT est de **+89 %** (4,2 ms → 7,9 ms), restant largement en deçà du seuil critique de 50 ms pour les applications IoT industrielles.

Abstract – *The growth of IoT in telecom operator environments generates massive and heterogeneous data flows exposed to increasing risks of interception, alteration and compromise. This final-year project, carried out in partnership with Tunisie Telecom and following Scrum methodology, addresses the end-to-end security of communication flows in a simulated IoT ecosystem. We designed and deployed a complete architecture including*

a PKI based on HashiCorp Vault, a Mosquitto MQTT broker with TLS 1.3/mTLS, an IoT fleet simulator, Suricata IDS, Wazuh SIEM and an ML anomaly detection pipeline. The RandomForest classifier achieves $F1=0.994$, $ROC-AUC=0.999$. TLS overhead on RTT is +89 % (4.2ms to 7.9ms), well within IoT industrial thresholds.

Table des matières

Dédicace	i
Remerciements	iii
Résumé	v
Liste des acronymes	xvi
1 Introduction Générale	1
1.1 Contexte et motivations	1
1.2 Problématique	1
1.3 Objectifs du projet	2
1.4 Plan du rapport	3
2 Cadre Général du Projet	4
2.1 Présentation de l'organisme d'accueil	4
2.1.1 Tunisie Telecom	4
2.1.2 Contexte du stage	4
2.1.3 Sujet du projet	4
2.2 Méthodologie de travail : Scrum	5
2.2.1 Choix de la méthodologie	5
2.2.2 Principes Scrum appliqués	5
2.2.3 Rôles Scrum	6
2.2.4 Artefacts Scrum	6
2.2.5 Cérémonies Scrum	6
2.3 Product Backlog	6
2.4 Planification des sprints	7
2.4.1 Sprint 0 – Cadrage (2 semaines)	8
2.4.2 Sprint 1 – Infrastructure réseau (3 semaines)	8
2.4.3 Sprint 2 – PKI et broker MQTT (3 semaines)	8
2.4.4 Sprint 3 – Simulation de la flotte (2 semaines)	9
2.4.5 Sprint 4 – Détection et analyse (3 semaines)	9

2.4.6	Sprint 5 – Synthèse et documentation (2 semaines)	9
2.5	Diagramme de cas d'utilisation général	9
3	État de l'Art	12
3.1	Protocoles de communication IoT	12
3.1.1	MQTT – Message Queuing Telemetry Transport	12
3.1.2	Comparatif des protocoles IoT	13
3.2	Sécurité des flux IoT	13
3.2.1	Vecteurs d'attaque principaux	13
3.2.2	TLS 1.3 et mTLS pour MQTT	13
3.3	Infrastructures à Clés Publiques pour l'IoT	15
3.3.1	Défis spécifiques à l'IoT	15
3.3.2	HashiCorp Vault comme PKI dynamique	15
3.4	Détection d'anomalies et SIEM	16
3.4.1	Suricata IDS	16
3.4.2	Wazuh SIEM	17
3.4.3	Machine Learning pour la détection d'anomalies IoT	17
3.5	Synthèse et positionnement	17
4	Architecture Générale du Système	18
4.1	Vue d'ensemble	18
4.2	Topologie réseau GNS3	19
4.2.1	Segmentation réseau	19
4.2.2	Composants réseau	20
4.3	Diagramme de déploiement	21
4.4	Composants applicatifs	22
4.4.1	Broker MQTT – Mosquitto	22
4.4.2	PKI – HashiCorp Vault	22
4.4.3	Fleet Simulator	22
4.4.4	Suricata IDS	22
4.4.5	Wazuh SIEM	22
4.5	Diagramme de composants logiciels	23
4.6	Flux de données E2E sécurisé	23
4.7	Choix technologiques	25
5	Infrastructure PKI et HashiCorp Vault	26
5.1	Conception de la hiérarchie de certification	26
5.1.1	Principes de conception	26
5.1.2	Paramètres cryptographiques	28
5.2	Déploiement de HashiCorp Vault	28

5.2.1	Configuration initiale	28
5.2.2	Création de la CA racine	29
5.2.3	Création de la CA intermédiaire	29
5.2.4	Définition des rôles d'émission	29
5.3	Émission automatique des certificats de dispositifs	29
5.4	Validation de la PKI	31
6	Configuration TLS/mTLS sur Mosquitto	32
6.1	Sécurisation du broker MQTT	32
6.1.1	Architecture de sécurité Mosquitto	32
6.1.2	Éléments de configuration principaux	32
6.1.3	Politiques ACL	33
6.2	Déploiement dans Docker/GNS3	33
6.3	Validation de la configuration TLS	34
6.3.1	Tests de connexion	34
6.3.2	Test fonctionnel MQTT	34
6.4	Analyse du handshake TLS 1.3	34
7	Simulation de la Flotte IoT	37
7.1	Présentation du dataset	37
7.1.1	Source des données	37
7.1.2	Distribution des classes	37
7.2	Architecture du simulateur de flotte	38
7.2.1	Conception	38
7.2.2	Structure logicielle	39
7.2.3	Processus de simulation	39
7.2.4	Paramètres de simulation	41
7.3	Injection des patterns d'attaque	41
7.4	Résultats de la simulation	41
8	Analyse Réseau avec Suricata	43
8.1	Déploiement de Suricata	43
8.1.1	Architecture de détection	43
8.1.2	Mode de capture	45
8.1.3	Activation du décodeur MQTT	45
8.2	Règles de détection personnalisées	45
8.3	Analyse des alertes	46
8.3.1	Volume d'alertes générées	46
8.3.2	Format des alertes EVE JSON	46

9	Intégration SIEM avec Wazuh	48
9.1	Présentation de Wazuh	48
9.2	Déploiement de Wazuh OVA	50
9.2.1	Choix du mode de déploiement	50
9.2.2	Architecture Wazuh	50
9.3	Configuration de l'ingestion Suricata	50
9.3.1	Logcollector	50
9.3.2	Règles de corrélation personnalisées	50
9.4	Visualisation dans le dashboard	51
9.4.1	Dashboard Wazuh – résultats	51
9.4.2	Distribution des alertes par niveau	52
10	Détection d'Anomalies par Machine Learning	53
10.1	Pipeline de traitement des données	53
10.1.1	Chargement et exploration	55
10.1.2	Prétraitement	56
10.2	Isolation Forest – Détection non supervisée	56
10.2.1	Principe et configuration	56
10.2.2	Résultats IsolationForest	57
10.3	Random Forest – Détection supervisée	58
10.3.1	Configuration et entraînement	58
10.3.2	Résultats Random Forest – Détection binaire	59
10.3.3	Classification multi-classe	60
10.4	Comparaison des modèles	61
11	Évaluation des Performances	63
11.1	Objectifs et méthodologie	63
11.1.1	Protocole de test	63
11.2	Latence de connexion	63
11.3	RTT des messages MQTT	64
11.4	Débit (Throughput)	65
11.5	Handshake TLS	66
11.6	Comparaison globale TCP vs TLS	67
11.7	Acceptabilité pour l'IoT	67
12	Conclusion et Perspectives	69
12.1	Synthèse des travaux	69
12.2	Objectifs atteints	69
12.3	Résultats quantitatifs clés	71
12.4	Contributions	71

12.5 Limitations	72
12.6 Perspectives	72
12.6.1 Court terme (3–6 mois)	72
12.6.2 Moyen terme (6–12 mois)	73
12.6.3 Long terme (> 12 mois)	73
Références bibliographiques	74
A Scripts et Code Source	76
A.1 Bootstrap de la PKI — Vault API	76
A.2 Émission du certificat Mosquitto	79
A.3 Génération des certificats devices	80
A.4 Génération du fichier ACL Mosquitto	81
A.5 Tests de connectivité réseau	81
A.6 Tests de performance MQTT	83
A.7 Simulateur de flotte IoT	84
A.8 Analyse des événements Suricata	85
B Fichiers de Configuration	87
B.1 Configuration MikroTik CHR	87
B.2 Configuration Mosquitto (TLS/mTLS)	89
B.3 Configuration Suricata (MQTT natif)	90
B.4 Règles Suricata personnalisées	91
B.5 Règles Wazuh personnalisées	92
B.6 Dockerfile du simulateur de flotte	93
B.7 Dockerfile Toolbox	94
B.8 Commandes de création des réseaux Docker	94
B.9 Variables d’environnement du projet	96

Table des figures

2.1	Planification des sprints du projet PFE	8
2.2	Diagramme de cas d'utilisation du système IoT sécurisé	10
3.1	Handshake TLS 1.3 en mode mTLS (diagramme de séquence)	14
3.2	Hiérarchie PKI – diagramme de classes (CA Racine, CA Intermédiaire, Rôles)	16
4.1	Architecture générale de l'écosystème IoT sécurisé	19
4.2	Topologie réseau détaillée (diagramme PlantUML)	20
4.3	Topologie réseau dans l'interface GNS3	21
4.4	Diagramme de déploiement – infrastructure physique et virtuelle	21
4.5	Diagramme de composants logiciels et interfaces	23
4.6	Flux E2E sécurisé – diagramme de séquence	24
5.1	Hiérarchie PKI – diagramme de classes	27
5.2	Séquence d'émission d'un certificat de dispositif via l'API Vault	30
6.1	Handshake TLS 1.3 mTLS – diagramme de séquence détaillé	35
7.1	Distribution des classes dans le dataset IoT	38
7.2	Diagramme de classes du simulateur de flotte IoT	38
7.3	Diagramme d'activité du simulateur de flotte IoT	40
8.1	Processus de détection Suricata – diagramme d'activité	44
8.2	Alertes Suricata visibles dans le dashboard Wazuh	47
9.1	Pipeline d'ingestion et de traitement Wazuh	49
9.2	Dashboard Wazuh affichant les alertes IoT/MQTT	51
10.1	Pipeline de détection d'anomalies par ML – diagramme d'activité	54
10.2	Distribution des classes dans le dataset (EDA)	55
10.3	Matrice de corrélation des features	55
10.4	Boxplots des features par type d'attaque	56
10.5	Matrice de confusion – IsolationForest	58

10.6 Distribution des scores d'anomalie – IsolationForest	58
10.7 Matrice de confusion et courbe ROC – Random Forest	59
10.8 Importance des features – Random Forest	60
10.9 Matrice de confusion multi-classe – Random Forest	61
10.10 Comparaison IsolationForest vs Random Forest	61
11.1 Comparaison de la latence de connexion TCP vs TLS	64
11.2 RTT des messages MQTT – TCP vs TLS par QoS	65
11.3 Throughput TCP vs TLS en fonction de la taille de payload	65
11.4 Décomposition des phases du handshake TLS 1.3	66
11.5 Comparaison synthétique des performances TCP vs TLS	67

Liste des tableaux

2.1	Répartition des rôles Scrum dans le projet	6
2.2	Cérémonies Scrum et leur application au projet	6
2.3	Product Backlog du projet	7
3.1	Comparatif des principaux protocoles de communication IoT	13
3.2	Positionnement par rapport aux travaux existants	17
4.1	Plan d’adressage de la topologie GNS3	20
4.2	Justification des choix technologiques	25
5.1	Paramètres cryptographiques de la PKI	28
5.2	Rôles d’émission PKI définis dans Vault	29
6.1	Directives principales de sécurité Mosquitto	33
6.2	Analyse temporelle du handshake TLS 1.3 en mTLS	36
7.1	Distribution des classes dans le dataset	37
7.2	Paramètres de la simulation	41
8.1	Règles Suricata personnalisées pour la détection des attaques IoT	45
8.2	Distribution des alertes Suricata par règle	46
9.1	Configuration de la VM Wazuh	50
9.2	Règles Wazuh personnalisées pour les alertes IoT	51
9.3	Distribution des alertes par niveau de sévérité Wazuh	52
10.1	Hyperparamètres de l’IsolationForest	57
10.2	Métriques IsolationForest (détection binaire normale/anomalie)	57
10.3	Hyperparamètres du Random Forest	59
10.4	Métriques Random Forest (détection binaire)	59
10.5	Comparaison finale des deux approches ML	62
11.1	Latence de connexion MQTT – TCP vs TLS 1.3	64
11.2	RTT des messages MQTT par QoS	64

11.3	Throughput par taille de payload (QoS 1)	65
11.4	Décomposition du temps de handshake TLS 1.3	66
11.5	Synthèse de l'impact TLS sur les métriques MQTT	67
11.6	Évaluation de l'acceptabilité des performances pour l'IoT	67
12.1	Bilan de réalisation des objectifs	70
12.2	Résultats quantitatifs principaux du projet	71
B.1	Variables d'environnement principales du projet	96

Liste des acronymes

IoT	Internet of Things – Internet des Objets
TLS	Transport Layer Security
mTLS	Mutual TLS – TLS mutuel (authentification bidirectionnelle)
DTLS	Datagram TLS
PKI	Public Key Infrastructure – Infrastructure à Clés Publiques
CA	Certificate Authority – Autorité de Certification
CSR	Certificate Signing Request
CRL	Certificate Revocation List
OCSP	Online Certificate Status Protocol
MQTT	Message Queuing Telemetry Transport
CoAP	Constrained Application Protocol
HTTP	HyperText Transfer Protocol
AMQP	Advanced Message Queuing Protocol
SIEM	Security Information and Event Management
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
DPI	Deep Packet Inspection
DoS	Denial of Service
DDoS	Distributed Denial of Service
MiTM	Man-in-the-Middle
ML	Machine Learning – Apprentissage Automatique
RF	Random Forest
IF	Isolation Forest
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
RTT	Round-Trip Time

QoS	Quality of Service
DMZ	DeMilitarized Zone
VLAN	Virtual Local Area Network
GNS3	Graphical Network Simulator 3
VM	Virtual Machine
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ACL	Access Control List
CN	Common Name
SAN	Subject Alternative Name
E2E	End-to-End
RSA	Rivest–Shamir–Adleman
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
OVA	Open Virtual Appliance
TT	Tunisie Telecom
FST	Faculté des Sciences de Tunis

Chapitre 1

Introduction Générale

1.1 Contexte et motivations

L’Internet des Objets (IoT) connaît une expansion exponentielle dans le secteur des télécommunications. Selon les estimations de la GSMA, le nombre de connexions IoT dépassera 25 milliards d’unités d’ici 2025, générant un volume de données sans précédent. Les opérateurs télécom comme *Tunisie Telecom* se trouvent au cœur de cette transformation numérique, en déployant des infrastructures capables d’absorber ces flux tout en garantissant leur confidentialité, leur intégrité et leur disponibilité.

Or, l’IoT présente des caractéristiques propres qui compliquent drastiquement la sécurisation : les dispositifs disposent souvent de ressources calculatoires limitées, leurs mises à jour de sécurité sont rares, et les protocoles de communication applicatifs (MQTT, CoAP) ont été conçus à l’origine sans mécanismes de sécurité obligatoires. Ces facteurs font des écosystèmes IoT des cibles de choix pour les attaquants, qui peuvent y déployer des *botnets*, des attaques par déni de service (Denial of Service ([DoS](#))), des interceptions (*Man-in-the-Middle*) ou encore des injections de données malveillantes.

Ce projet de fin d’études, réalisé au sein de *Tunisie Telecom*, répond à un besoin concret et stratégique : définir, implémenter et valider une architecture de sécurité complète pour des flux MQTT en environnement IoT simulé.

1.2 Problématique

La problématique centrale de ce travail peut se formuler ainsi :

Problématique

Comment sécuriser de bout en bout les flux de communication d'un écosystème IoT simulé, depuis l'authentification des dispositifs jusqu'à la détection automatisée des comportements anormaux, tout en maintenant des performances acceptables pour les applications temps-réel ?

Cette problématique se décompose en quatre sous-questions directrices :

1. Comment établir une chaîne de confiance cryptographique pour des milliers de dispositifs IoT hétérogènes ?
2. Comment configurer le protocole MQTT pour exiger une authentification mutuelle TLS (mTLS) sans dégrader significativement les performances ?
3. Comment détecter en temps réel les attaques réseau et les comportements anormaux dans un flux MQTT chiffré ?
4. Dans quelle mesure les approches de Machine Learning non supervisé sont-elles viables pour la détection d'anomalies dans des contextes IoT réels ?

1.3 Objectifs du projet

Les objectifs opérationnels de ce travail sont les suivants :

- **O1 – Infrastructure PKI** : Déployer une hiérarchie de certification (CA racine, CA intermédiaire) basée sur HashiCorp Vault pour émettre et gérer automatiquement les certificats de dispositifs et de services.
- **O2 – Broker sécurisé** : Configurer un broker Mosquitto en TLS 1.3 strict avec mTLS obligatoire et politiques d'accès par topic.
- **O3 – Simulation réaliste** : Reproduire le comportement d'une flotte de 50 capteurs IoT en rejouant un dataset de 76 000 événements réels couvrant sept types d'attaques.
- **O4 – Analyse réseau** : Déployer Suricata avec des règles IDS personnalisées pour détecter les anomalies dans les flux MQTT.
- **O5 – SIEM** : Intégrer Wazuh pour la collecte, la corrélation et l'alerte des événements de sécurité.
- **O6 – Détection ML** : Comparer IsolationForest (non supervisé) et RandomForest (supervisé) pour la détection d'anomalies.
- **O7 – Performance** : Quantifier l'overhead TLS/mTLS sur les métriques MQTT clés (latence, débit, RTT).

1.4 Plan du rapport

Ce rapport est structuré en douze chapitres :

Chapitre 1 Introduction générale, problématique et objectifs.

Chapitre 2 Cadre général du projet : présentation de l'organisme d'accueil (*Tunisie Telecom*), méthodologie Scrum et planification.

Chapitre 3 État de l'art : protocoles IoT, sécurité des flux, PKI, ML pour la détection d'anomalies.

Chapitre 4 Architecture générale du système : topologie réseau, composants, flux E2E.

Chapitre 5 Infrastructure PKI et déploiement de HashiCorp Vault.

Chapitre 6 Configuration TLS/mTLS sur le broker Mosquitto.

Chapitre 7 Simulation de la flotte IoT et injection des patterns d'attaque.

Chapitre 8 Analyse réseau avec Suricata IDS.

Chapitre 9 Intégration SIEM avec Wazuh.

Chapitre 10 Détection d'anomalies par Machine Learning.

Chapitre 11 Évaluation des performances.

Chapitre 12 Conclusion et perspectives.

Les annexes A et B regroupent l'intégralité des scripts d'automatisation et des configurations techniques des composants déployés.

Chapitre 2

Cadre Général du Projet

2.1 Présentation de l'organisme d'accueil

2.1.1 Tunisie Telecom

Tunisie Telecom, fondée en 1995 et privatisée partiellement en 2006, est l'opérateur historique de télécommunications en Tunisie. Avec plus de 7 millions d'abonnés fixes et mobiles, elle fournit des services de téléphonie, d'accès Internet haut débit (ADSL, fibre optique) et de solutions entreprise.

Tunisie Telecom dispose d'une infrastructure réseau couvrant l'ensemble du territoire tunisien, incluant :

- Un réseau cœur IP/MPLS interconnectant les différentes régions.
- Des data centers hébergeant les services cloud et d'hébergement.
- Une plate-forme IoT en cours de déploiement pour les projets Smart City et industrie 4.0.

2.1.2 Contexte du stage

Le stage s'est déroulé au sein de l'équipe **Sécurité des Réseaux** de *Tunisie Telecom*, sous la supervision de **M. Moez KHLIFI**. L'équipe est en charge de la surveillance, de l'audit et du durcissement des infrastructures télécom de l'opérateur. Le projet PFE s'inscrit dans la stratégie de *Tunisie Telecom* visant à évaluer les solutions de sécurité adaptées aux déploiements IoT à grande échelle.

2.1.3 Sujet du projet

Le sujet confié porte sur la **sécurisation des flux de communication pour un écosystème IoT simulé**. Il s'agit de concevoir, déployer et valider une architecture de sécurité de bout en bout (E2E) couvrant :

- L’authentification des dispositifs via une PKI dynamique.
- Le chiffrement des communications MQTT par TLS 1.3 avec authentification mutuelle (mTLS).
- La détection d’intrusions réseau par un IDS (Suricata).
- L’agrégation et la corrélation des événements de sécurité par un SIEM (Wazuh).
- La détection d’anomalies comportementales par des algorithmes de Machine Learning.

2.2 Méthodologie de travail : Scrum

2.2.1 Choix de la méthodologie

Pour mener à bien ce projet, nous avons adopté la méthodologie **Scrum**, un cadre de travail agile itératif et incrémental. Ce choix est motivé par :

- La nature exploratoire du projet, nécessitant des ajustements fréquents de l’architecture.
- La nécessité de livrer des incréments fonctionnels à chaque itération pour validation par l’encadrant.
- La durée limitée du stage (environ 4 mois), imposant une gestion rigoureuse du temps et des priorités.

2.2.2 Principes Scrum appliqués

Scrum repose sur trois piliers fondamentaux : la **transparence**, l’**inspection** et l’**adaptation**. Dans le cadre de ce PFE, ces principes se sont traduits par :

Transparence Un journal de bord (`JOURNAL.md`) consigne quotidiennement les avancées, les blocages et les décisions techniques. L’intégralité du code source est versionné sous Git.

Inspection Des points d’avancement hebdomadaires avec l’encadrante universitaire (Mme. ELLOUZE) et des revues bimensuelles avec l’encadrant entreprise (M. KHLIFI) permettent de valider les livrables.

Adaptation Le Product Backlog est réajusté à chaque fin de sprint en fonction des résultats obtenus et des nouvelles contraintes identifiées.

2.2.3 Rôles Scrum

TABLE 2.1 – Répartition des rôles Scrum dans le projet

Rôle	Personne	Responsabilité
Product Owner	M. Moez KHLIFI	Définition des besoins métier, priorisation du backlog, validation des livrables
Scrum Master	Mme. ELLOUZE NOURHENE	Suivi de l'avancement, levée des blocages, revue académique
Équipe de développement	Amine GHANNOUCHI	Conception, implémentation, tests, documentation

2.2.4 Artefacts Scrum

Product Backlog Liste priorisée de toutes les fonctionnalités et tâches techniques du projet, découpée en *user stories* et tâches techniques (voir tableau 2.3).

Sprint Backlog Sous-ensemble du Product Backlog sélectionné pour chaque sprint, avec les critères d'acceptation associés.

Incrément Livrable fonctionnel et testé produit à la fin de chaque sprint (composant déployé, tests validés, documentation associée).

2.2.5 Cérémonies Scrum

TABLE 2.2 – Cérémonies Scrum et leur application au projet

Cérémonie	Fréquence	Participants
Sprint Planning	Début de sprint	PO, SM, Dev
Daily Stand-up	Quotidien (journal)	Dev (auto-suivi)
Sprint Review	Fin de sprint	PO, SM, Dev
Sprint Retrospective	Fin de sprint	SM, Dev

2.3 Product Backlog

Le Product Backlog initial a été établi lors du Sprint 0 (cadrage) et affiné itérativement. Le tableau 2.3 présente les principales *user stories* et tâches techniques.

TABLE 2.3 – Product Backlog du projet

ID	Priorité	Description	Sprint
US-01	Haute	Installer et configurer GNS3 avec VMware	1
US-02	Haute	Créer la topologie réseau avec VLANs	1
US-03	Haute	Déployer pfSense et MikroTik CHR	1
US-04	Haute	Valider la connectivité inter-VLAN	1
US-05	Haute	Déployer Vault et activer le moteur PKI	2
US-06	Haute	Créer la hiérarchie CA racine + intermédiaire	2
US-07	Haute	Configurer Mosquitto en TLS 1.3/mTLS	2
US-08	Haute	Automatiser l'émission des certificats	2
US-09	Moyenne	Développer le simulateur de flotte IoT	3
US-10	Moyenne	Injecter le dataset CSV (76 000 événements)	3
US-11	Moyenne	Déployer Suricata avec règles MQTT	4
US-12	Moyenne	Déployer Wazuh OVA et configurer l'ingestion	4
US-13	Moyenne	Développer le pipeline ML (IF + RF)	4
US-14	Basse	Réaliser les tests de performance TLS	4
US-15	Basse	Rédiger le rapport et préparer la soutenance	5

2.4 Planification des sprints

Le projet est organisé en **six sprints** d'une durée de deux à trois semaines chacun, comme illustré par la figure 2.1.

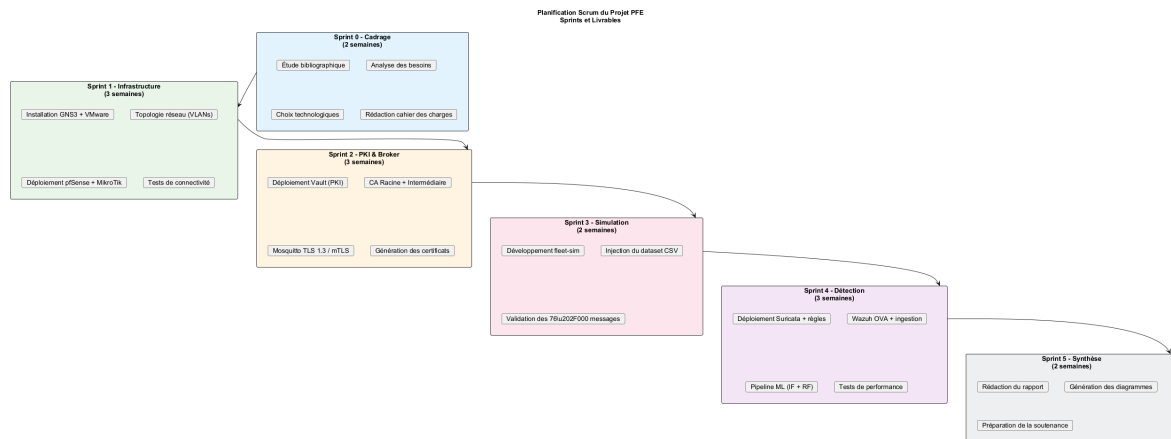


FIGURE 2.1 – Planification des sprints du projet PFE

2.4.1 Sprint 0 – Cadrage (2 semaines)

- Analyse des besoins fonctionnels et non fonctionnels.
- Étude bibliographique : protocoles IoT, TLS/mTLS, PKI, SIEM, ML.
- Choix des technologies et outils (GNS3, Vault, Mosquitto, Suricata, Wazuh, scikit-learn).
- Rédaction du cahier des charges et validation par le Product Owner.

Livrable : Cahier des charges validé, architecture cible définie.

2.4.2 Sprint 1 – Infrastructure réseau (3 semaines)

- Installation de GNS3 et VMware Workstation Pro sur la machine hôte.
- Création de la topologie réseau avec trois VLANs (IoT, Services, Management).
- Déploiement de pfSense CE comme pare-feu périphérique.
- Configuration de MikroTik CHR pour le routage inter-VLAN.
- Tests de connectivité et validation des règles de firewall.

Livrable : Topologie réseau opérationnelle dans GNS3, tests de connectivité validés.

2.4.3 Sprint 2 – PKI et broker MQTT (3 semaines)

- Déploiement de HashiCorp Vault en conteneur Docker.
- Création de la CA racine et de la CA intermédiaire.
- Définition des rôles d'émission (services et dispositifs IoT).
- Configuration du broker Mosquitto en TLS 1.3 strict avec mTLS obligatoire.
- Automatisation de l'émission des 50 certificats de dispositifs.

- Validation de la chaîne de confiance complète.

Livvable : PKI fonctionnelle, broker MQTT sécurisé, 50 certificats émis.

2.4.4 Sprint 3 – Simulation de la flotte (2 semaines)

- Développement du simulateur de flotte IoT en Python (multi-thread).
- Chargement et partitionnement du dataset CSV entre 50 dispositifs simulés.
- Injection des 76 000 messages MQTT en mTLS.
- Validation de la capture réseau par Suricata.

Livvable : 76 000 messages publiés en mTLS, capture réseau disponible.

2.4.5 Sprint 4 – Détection et analyse (3 semaines)

- Déploiement de Suricata v7.0 avec le décodeur MQTT natif.
- Rédaction de quatre règles IDS personnalisées (DoS, injection, scanning, MiTM).
- Déploiement de Wazuh v4.7 (OVA) et configuration de l'ingestion des alertes Suricata.
- Création des règles de corrélation Wazuh personnalisées.
- Développement du pipeline ML : IsolationForest + RandomForest.
- Exécution des tests de performance (latence, RTT, débit).

Livvable : 51 alertes Suricata, dashboard Wazuh, modèles ML entraînés (F1 RF = 0,994).

2.4.6 Sprint 5 – Synthèse et documentation (2 semaines)

- Rédaction du rapport de PFE.
- Génération des diagrammes UML (PlantUML).
- Préparation de la présentation de soutenance.
- Revue finale avec les encadrants.

Livvable : Rapport final, support de soutenance.

2.5 Diagramme de cas d'utilisation général

La figure 2.2 présente le diagramme de cas d'utilisation global du système, identifiant les trois acteurs principaux et leurs interactions.

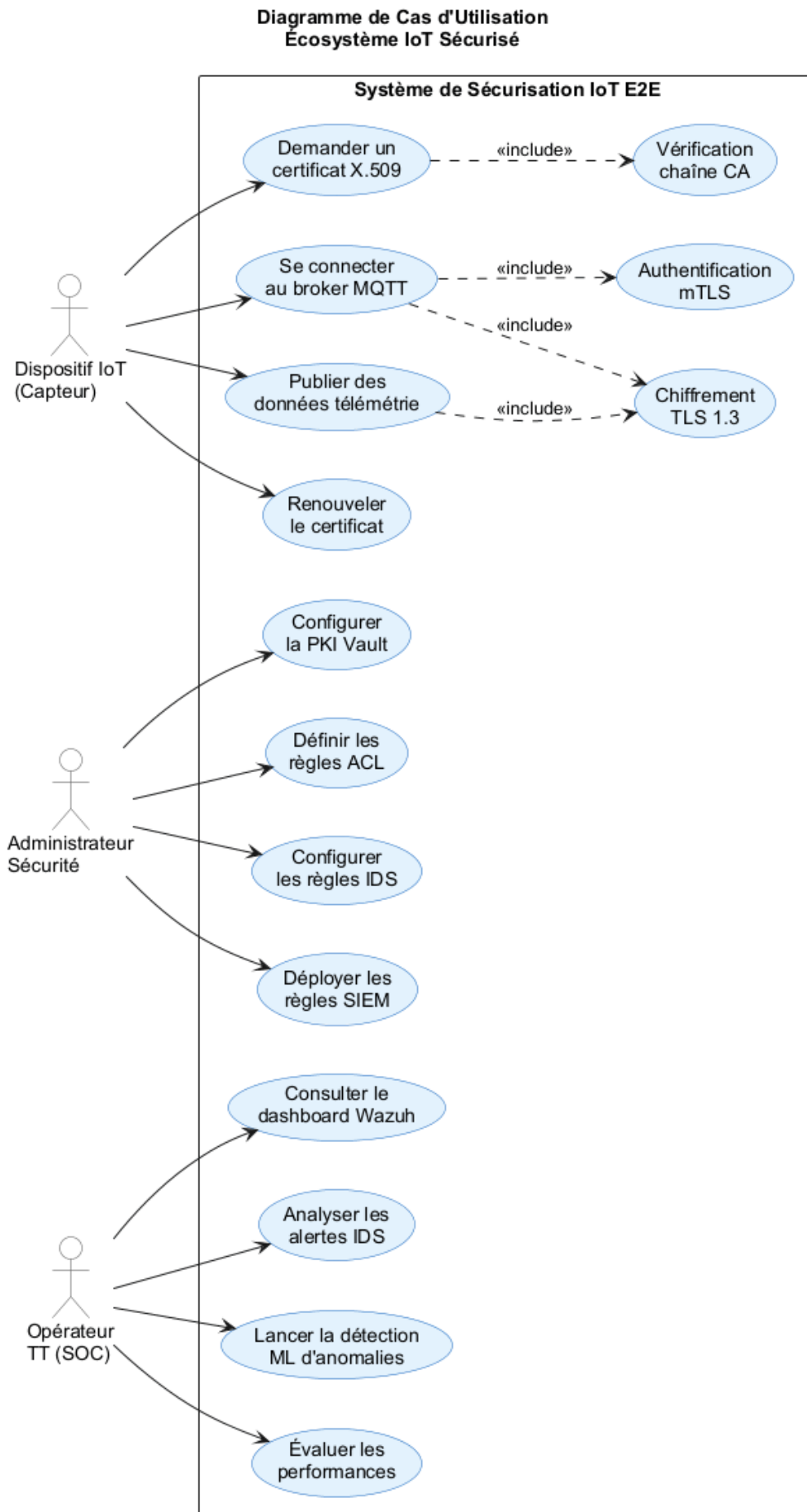


FIGURE 2.2 – Diagramme de cas d'utilisation du système IoT sécurisé

Les trois acteurs identifiés sont :

Dispositif IoT Demande un certificat, établit une connexion mTLS, publie des données de télémétrie.

Administrateur Gère la PKI, configure les politiques de sécurité, surveille les performances.

Analyste SOC Consulte les alertes Suricata/Wazuh, analyse les anomalies ML, gère les incidents.

Chapitre 3

État de l’Art

3.1 Protocoles de communication IoT

3.1.1 MQTT – Message Queuing Telemetry Transport

MQTT est un protocole de messagerie *publish/subscribe* conçu pour les réseaux à faible bande passante et les dispositifs à ressources limitées. Initialement développé par IBM en 1999, il est devenu la norme **ISO/IEC 20922 :2016** et le standard de facto pour l’IoT industriel [1].

Son architecture repose sur trois acteurs :

- Le **broker** (courtier), nœud central chargé du routage des messages.
- Les **publishers**, dispositifs émettant des messages sur des *topics*.
- Les **subscribers**, services consommant des messages en s’abonnant à des topics.

MQTT supporte trois niveaux de qualité de service (Quality of Service (**QoS**)) : QoS 0 (*at most once*), QoS 1 (*at least once*) et QoS 2 (*exactly once*). Dans notre projet, QoS 1 est utilisé pour l’équilibre fiabilité/performance.

Faiblesses natives : Sans configuration explicite, MQTT v3.1.1 transmet les données en clair sur TCP port 1883 et n’effectue aucune vérification d’identité. MQTT v5 améliore la situation mais ne rend pas TLS obligatoire.

3.1.2 Comparatif des protocoles IoT

TABLE 3.1 – Comparatif des principaux protocoles de communication IoT

Protocole	Transport	Modèle	QoS	Sécu. native	Usage typique
MQTT 5	TCP	Pub/Sub	0-1-2	TLS optionnel	IoT, télémétrie
CoAP	UDP	Req/Resp	Oui	DTLS optionnel	Contraints (micro-contrôleurs)
AMQP 1.0	TCP	Pub/Sub	Oui	TLS + SASL	Entreprise, finance
HTTP/2	TCP	Req/Resp	N/A	TLS (HTTPS)	APIs REST IoT
WebSocket	TCP	Full-duplex	N/A	TLS	Dashboards, temps-réel

3.2 Sécurité des flux IoT

3.2.1 Vecteurs d'attaque principaux

L'ENISA [2] et le MITRE ATT&CK ICS framework identifient plusieurs catégories d'attaques ciblant les écosystèmes IoT :

Interception (MiTM) Absence de chiffrement ou certificats non vérifiés permettant l'écoute passive ou l'injection active.

Usurpation d'identité Connexion au broker avec des identifiants volés ou sans authentification forte.

DoS / DDoS Inondation du broker avec des milliers de connexions ou de publications pour saturer les ressources [3].

Injection de données Publication de valeurs malveillantes sur les topics de commande pour affecter les actionneurs.

Replay Rejeu de messages chiffrés capturés pour déclencher des actions non autorisées.

3.2.2 TLS 1.3 et mTLS pour MQTT

TLS 1.3 (RFC 8446 [4], 2018) est la version recommandée pour les connexions MQTT sécurisées. Ses principales avancées par rapport à TLS 1.2 sont :

- **0-RTT resumption** pour les connexions répétées (sur même session).
- Suppression des algorithmes faibles (RSA key exchange, RC4, MD5/SHA-1).
- **Forward Secrecy** obligatoire via ECDHE.

- Handshake réduit à **1 RTT** contre 2 RTT pour TLS 1.2.

Le **mTLS** (*mutual TLS*) étend le handshake standard en exigeant que le client présente également un certificat signé par la CA de confiance. Cela permet d'authentifier à la fois le serveur *et* le client, ce qui est critique pour des dispositifs IoT dont on veut contrôler l'identité.

La figure 3.1 illustre le processus de handshake TLS 1.3 en mode mTLS, incluant l'échange bidirectionnel de certificats.

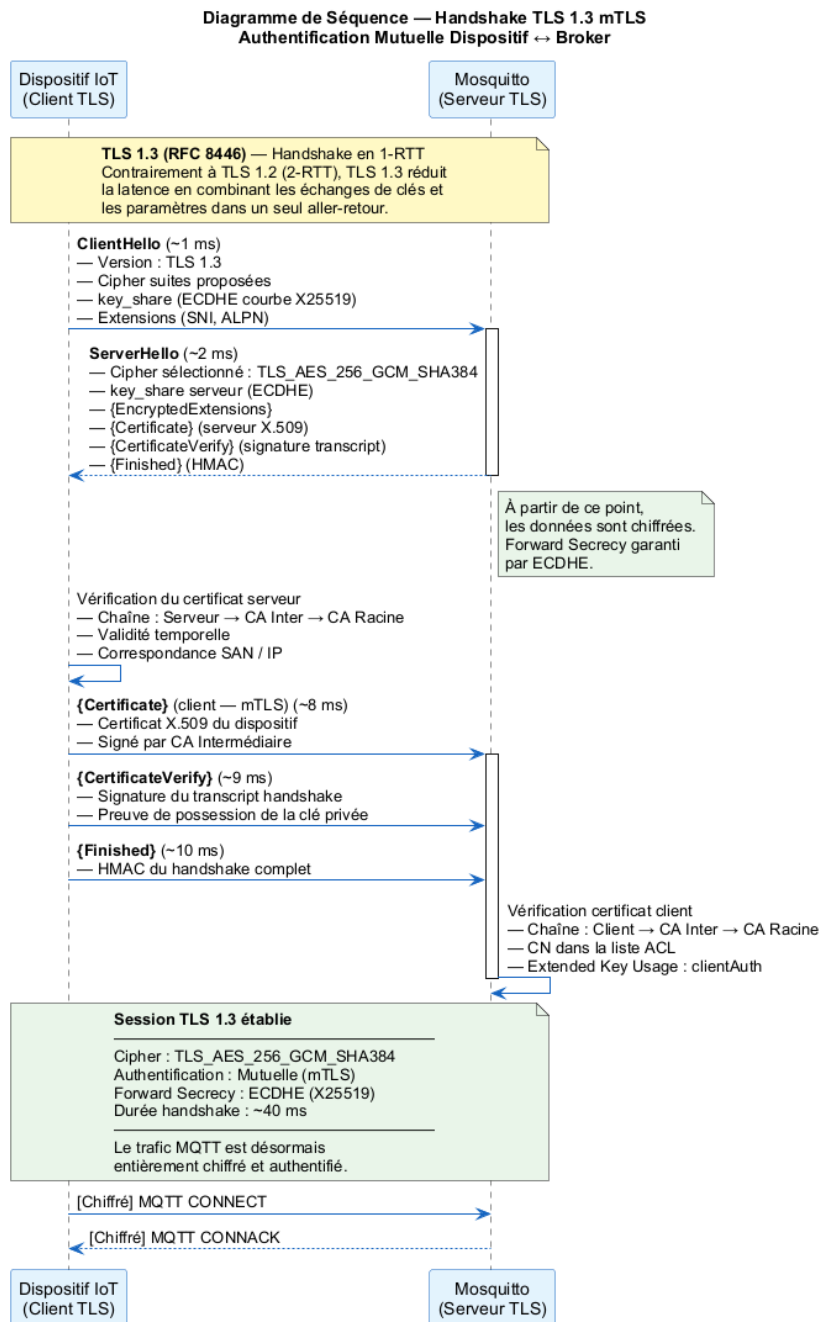


FIGURE 3.1 – Handshake TLS 1.3 en mode mTLS (diagramme de séquence)

3.3 Infrastructures à Clés Publiques pour l'IoT

3.3.1 Défis spécifiques à l'IoT

La gestion des certificats dans un écosystème IoT présente des défis uniques [5] :

- **Volume** : Des milliers à millions de dispositifs, chacun nécessitant un certificat unique.
- **Durée de vie courte** : Pour limiter l'exposition en cas de compromission.
- **Rotation automatisée** : Les dispositifs ne peuvent pas renouveler manuellement leurs certificats.
- **Révocation** : Les CRL ou OCSP doivent être accessibles même sur réseaux contraints.

3.3.2 HashiCorp Vault comme PKI dynamique

HashiCorp Vault est une solution open-source de gestion des secrets et de PKI dynamique [6]. Son moteur `pki` permet :

- L'émission automatisée de certificats X.509 via API REST ou CLI.
- La définition de **rôles** avec des contraintes (TTL, CN autorisés, IP SAN).
- L'intégration avec des orchestrateurs (Kubernetes, Nomad) ou des scripts shell.
- La révocation immédiate et la génération automatique de CRL.

La figure 3.2 présente le modèle de hiérarchie PKI à deux niveaux retenu dans notre architecture.

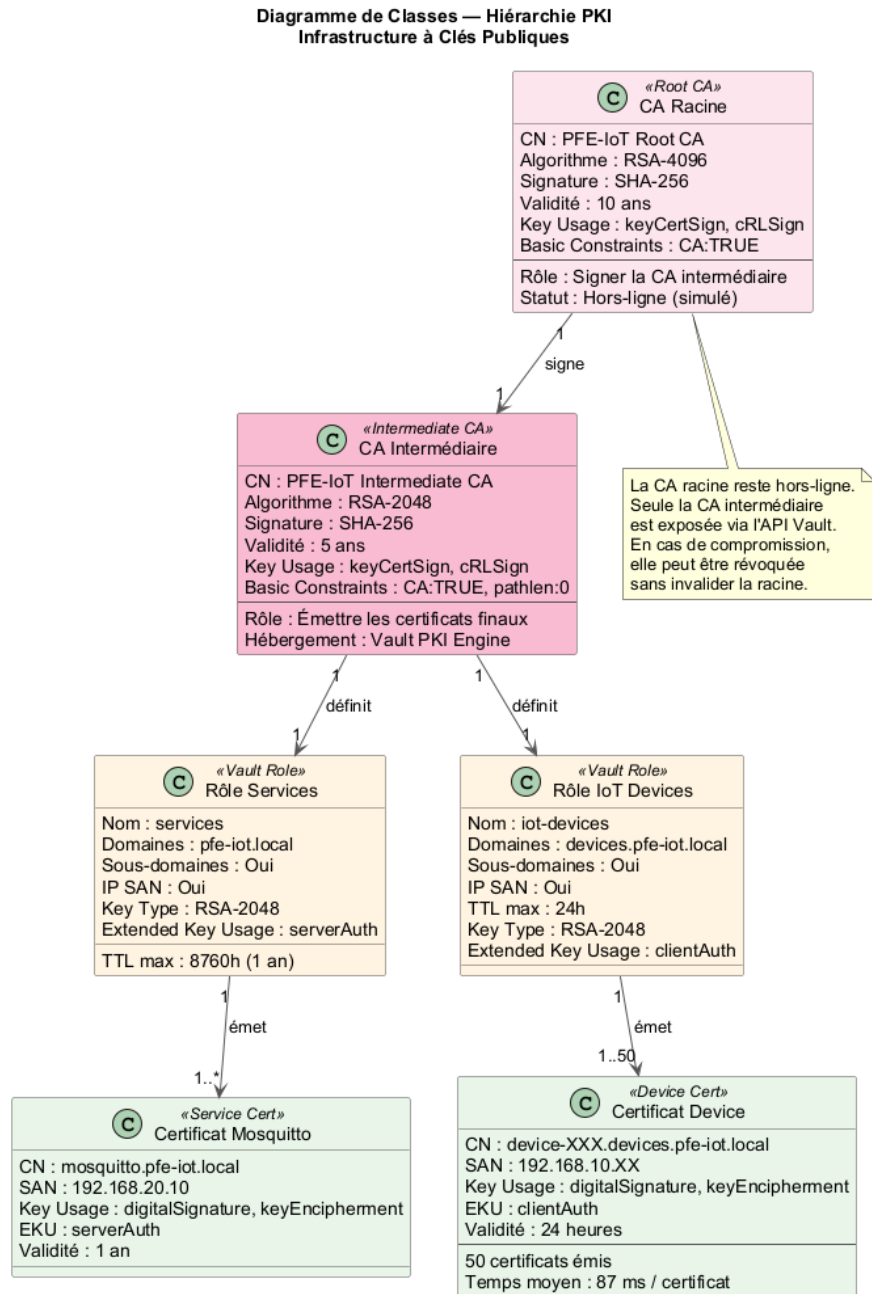


FIGURE 3.2 – Hiérarchie PKI – diagramme de classes (CA Racine, CA Intermédiaire, Rôles)

3.4 Détection d'anomalies et SIEM

3.4.1 Suricata IDS

Suricata est un moteur IDS/IPS open-source multi-thread supportant l'inspection en profondeur des paquets (DPI) [7]. Il supporte nativement le protocole MQTT depuis sa version 6.0 (`app-layer-protocol: mqtt`), ce qui permet d'écrire des règles spécifiques aux topics et aux payloads MQTT.

3.4.2 Wazuh SIEM

Wazuh est une plateforme SIEM open-source issue du fork d'OSSEC [8]. Elle offre :

- Un agent léger déployable sur Linux, Windows, macOS et conteneurs.
- Un moteur de règles configurable avec corrélation multi-sources.
- L'intégration native avec OpenSearch/Elasticsearch pour la visualisation.
- Des modules spécialisés pour Suricata, Docker, Kubernetes, AWS CloudTrail.

3.4.3 Machine Learning pour la détection d'anomalies IoT

Deux approches complémentaires ont été retenues :

IsolationForest [9] est un algorithme non supervisé basé sur la construction d'arbres d'isolation aléatoires. Les points anormaux, étant rares et distincts, sont isolés en un nombre minimal de partitions. Son avantage majeur est l'absence de nécessité d'étiquettes : il peut détecter des attaques inconnues.

Random Forest [10] est un ensemble d'arbres de décision entraîné en mode supervisé. Il exploite les étiquettes disponibles pour apprendre les signatures spécifiques de chaque type d'attaque, atteignant des performances proches de la perfection sur des datasets équilibrés.

3.5 Synthèse et positionnement

TABLE 3.2 – Positionnement par rapport aux travaux existants

Solution	PKI complète	mTLS MQTT	IDS	ML détection
AWS IoT Core	✓	✓	~	~
Azure IoT Hub	✓	✓	~	✓
Eclipse Mosquitto (base)	×	Optionnel	×	×
Travaux académiques	Partiel	Partiel	Partiel	Isolé
Cette architecture	✓	✓	✓	✓

Notre contribution se distingue par l'intégration verticale complète, du certificat dispositif jusqu'à la détection ML, dans un environnement simulé reproductible et entièrement open-source.

Chapitre 4

Architecture Générale du Système

4.1 Vue d'ensemble

L'architecture retenue pour ce projet est basée sur une approche **défense en profondeur** (*defense in depth*) : chaque couche de l'écosystème dispose de ses propres mécanismes de sécurité, de sorte qu'une compromission partielle ne compromet pas l'ensemble du système.

La figure [4.1](#) présente la vue d'ensemble de l'architecture, organisée en cinq zones de sécurité distinctes.

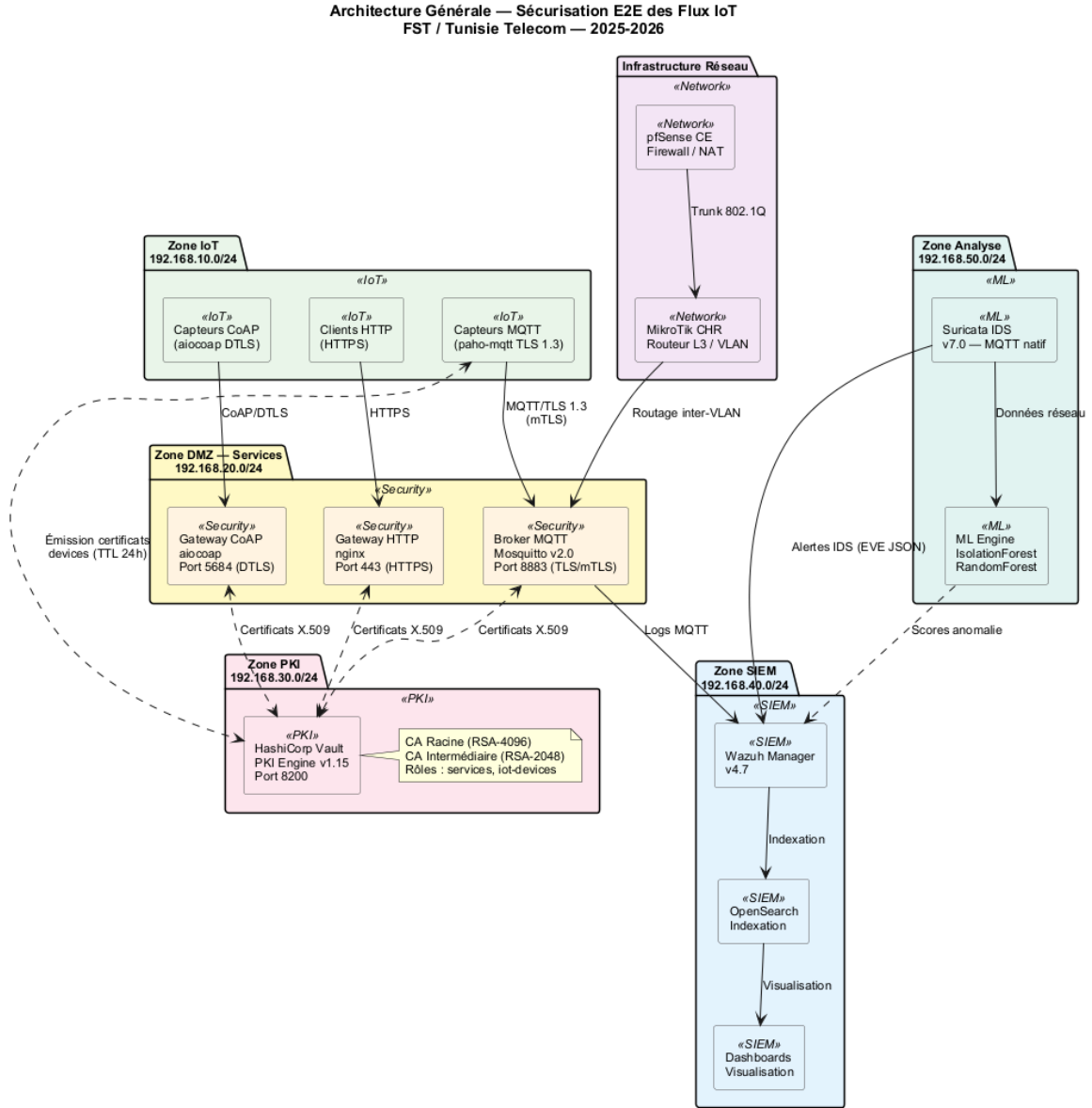


FIGURE 4.1 – Architecture générale de l'écosystème IoT sécurisé

4.2 Topologie réseau GNS3

La simulation réseau est réalisée sous **GNS3** (Graphical Network Simulator 3) avec VMware Workstation Pro comme hyperviseur [11]. L'environnement comprend l'ensemble des composants décrits ci-après.

4.2.1 Segmentation réseau

La topologie est segmentée en trois VLANs isolés par un routeur MikroTik CHR, conformément au plan d'adressage décrit dans le tableau 4.1.

TABLE 4.1 – Plan d’adressage de la topologie GNS3

Réseau	CIDR	Composants	Rôle
IoT Devices	192.168.10.0/24	fleet-simulator, capteurs	Zone dispositifs – trafic MQTT
Services	192.168.20.0/24	Mosquitto, Vault, Suricata	Zone services sécurisés
Management	192.168.30.0/24	Wazuh OVA	Zone administration et SIEM

La figure 4.2 illustre la topologie réseau détaillée avec les interconnexions entre VLANs, les ports et les adresses IP de chaque composant.

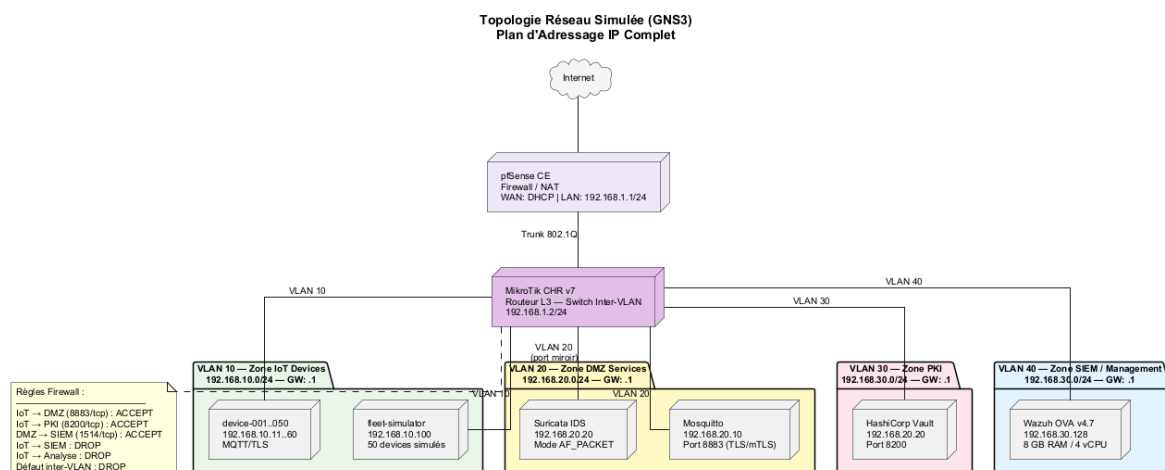


FIGURE 4.2 – Topologie réseau détaillée (diagramme PlantUML)

4.2.2 Composants réseau

Un routeur **MikroTik CHR** (RouterOS v7) assure le routage inter-VLAN et implémente les règles de pare-feu :

- Autorisation explicite du port 8883 (MQTT/TLS) de la zone IoT vers la zone Services.
- Blocage de tout trafic direct IoT → Management.
- NAT masquerade pour l’accès Internet (mises à jour, synchronisation temporelle).

Un pare-feu **pfSense CE** protège le périmètre externe de l’environnement simulé.

La capture d’écran de la figure 4.3 montre la topologie telle qu’affichée dans l’interface GNS3.

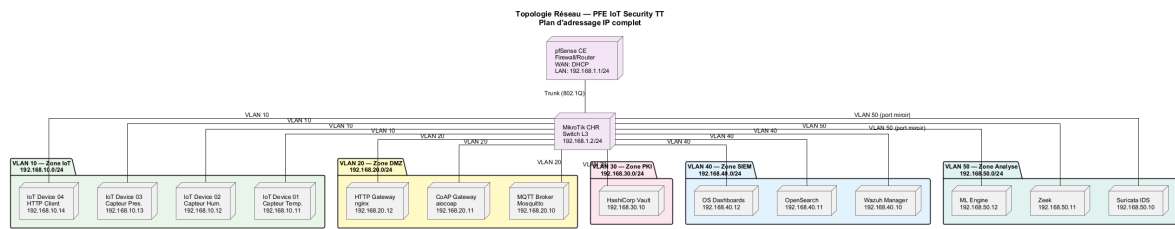


FIGURE 4.3 – Topologie réseau dans l'interface GNS3

4.3 Diagramme de déploiement

La figure 4.4 présente le diagramme de déploiement UML montrant l'ensemble des nœuds physiques et virtuels de l'infrastructure, ainsi que les artefacts logiciels déployés sur chacun.

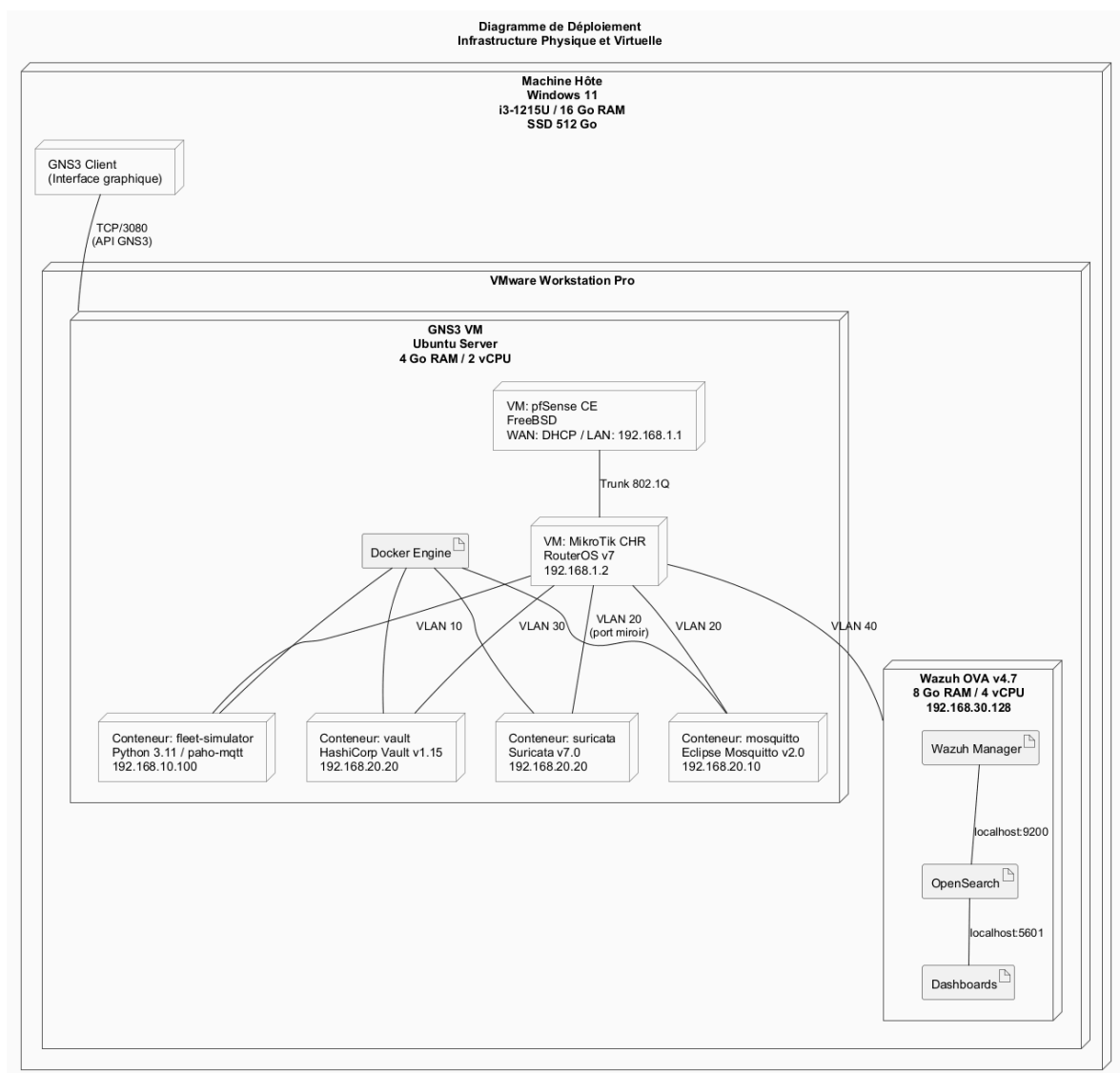


FIGURE 4.4 – Diagramme de déploiement – infrastructure physique et virtuelle

L'infrastructure s'appuie sur une machine hôte Windows 11 (Intel i3-1215U, 16 Go de RAM) hébergeant VMware Workstation Pro. La VM GNS3 (Ubuntu Server, 4 Go de RAM) exécute le moteur Docker et les conteneurs applicatifs. La VM Wazuh (OVA, 8 Go de RAM) est déployée séparément pour des raisons de performance.

4.4 Composants applicatifs

4.4.1 Broker MQTT – Mosquitto

Eclipse Mosquitto v2.0 [12] est déployé en conteneur Docker sur la zone Services (192.168.20.10). Il est configuré avec :

- TLS 1.3 strict (port 8883 uniquement, port 1883 désactivé).
- mTLS obligatoire (`require_certificate true`).
- ACL basée sur le **Common Name** du certificat client.

La configuration complète du broker est détaillée en Annexe B (section B.2).

4.4.2 PKI – HashiCorp Vault

Vault v1.15 [6] est déployé sur 192.168.20.20. Il héberge :

- Un moteur PKI racine (`pki`) avec une CA hors-ligne simulée.
- Un moteur PKI intermédiaire (`pki_int`) émettant les certificats de services et de dispositifs.
- Des politiques Vault contrôlant l'accès à l'API d'émission de certificats.

4.4.3 Fleet Simulator

Le simulateur de flotte est un conteneur Python (192.168.10.X) jouant le dataset CSV de 76 000 événements. Chaque dispositif simulé possède un certificat unique émis par Vault. La conception détaillée est présentée au chapitre 7.

4.4.4 Suricata IDS

Suricata v7.0 [7] est déployé en mode *AF_PACKET* sur le segment Services, avec une copie du trafic réseau via un switch GNS3 en mode hub.

4.4.5 Wazuh SIEM

Wazuh v4.7 [8] est déployé sous forme d'OVA VMware sur 192.168.30.128. Il ingère les alertes Suricata, les logs du broker Mosquitto et les journaux système des conteneurs.

4.5 Diagramme de composants logiciels

La figure 4.5 détaille les composants logiciels et leurs interfaces de communication, avec les protocoles et ports utilisés entre chaque élément.

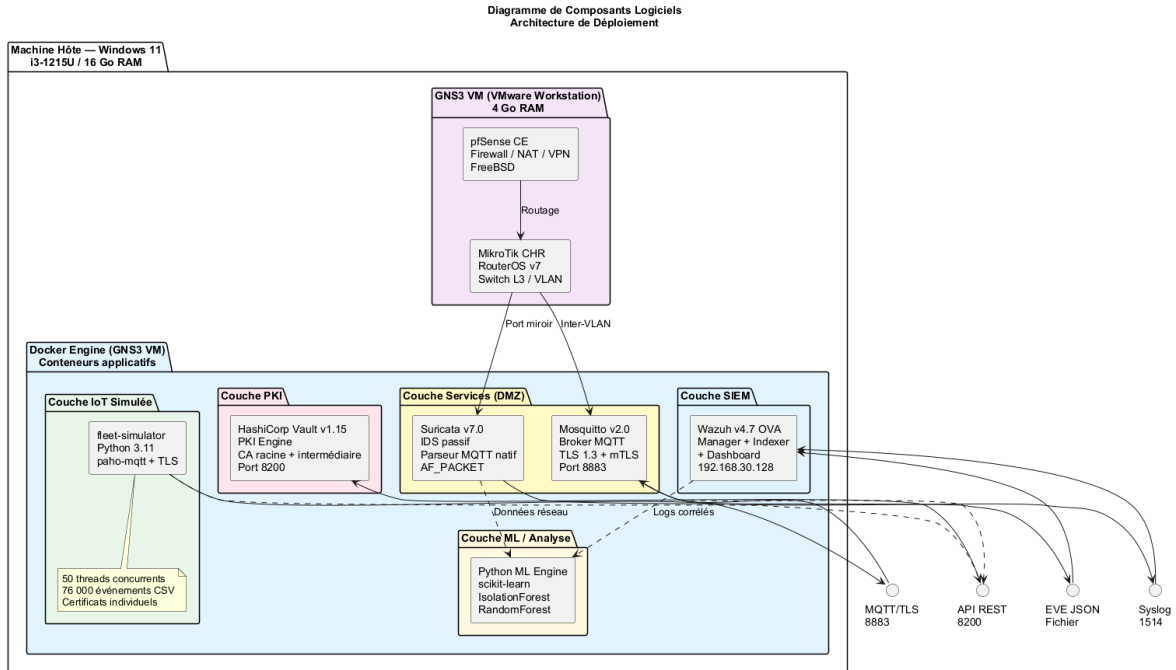


FIGURE 4.5 – Diagramme de composants logiciels et interfaces

4.6 Flux de données E2E sécurisé

Le flux de communication de bout en bout suit les étapes décrites dans la section suivante et est illustré par le diagramme de séquence de la figure 4.6.

1. Le dispositif IoT demande un certificat à Vault via l'API REST (authentification par token).
2. Vault émet un certificat X.509 signé par la CA intermédiaire, valide 24 heures.
3. Le dispositif établit une connexion TLS 1.3 avec Mosquitto (port 8883), en présentant son certificat.
4. Mosquitto valide la chaîne de certification et autorise la connexion si le CN correspond à la politique ACL.
5. Les messages MQTT sont publiés sur les topics autorisés et transités vers les subscribers.
6. Suricata inspecte le trafic réseau et génère des alertes JSON dans `eve.json`.
7. Wazuh ingère les alertes Suricata et les corrèle avec d'autres événements.

8. Le pipeline ML analyse les métriques de communication pour détecter des anomalies.

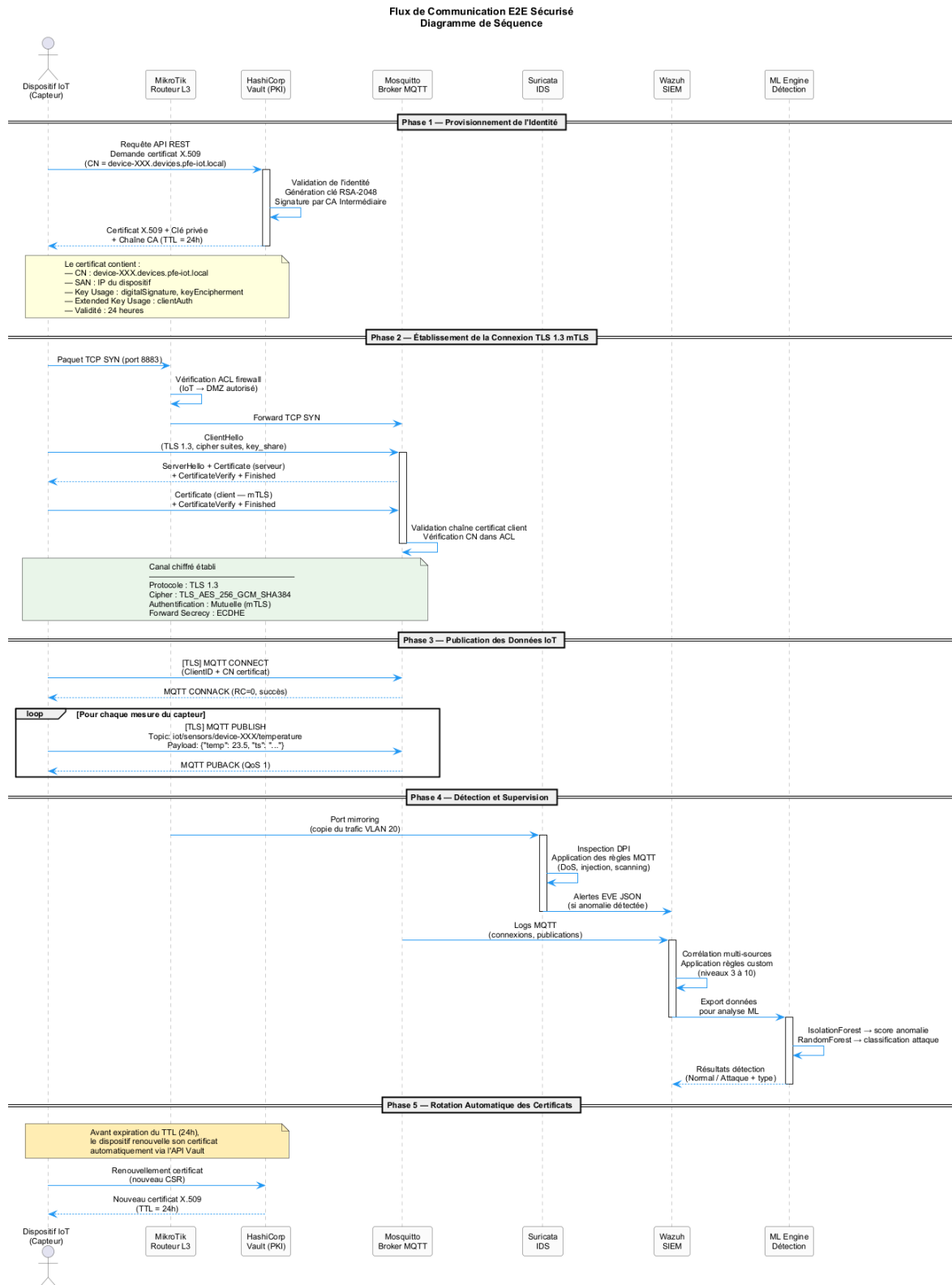


FIGURE 4.6 – Flux E2E sécurisé – diagramme de séquence

4.7 Choix technologiques

Les principaux choix techniques ont été motivés par les critères présentés dans le tableau 4.2.

TABLE 4.2 – Justification des choix technologiques

Composant	Technologie	Justification
Simulation réseau	GNS3	Standard académique, support Docker natif, reproductible
Routeur	MikroTik CHR	Utilisé en production chez TT, RouterOS complet
Broker MQTT	Mosquitto	Open-source, TLS 1.3 natif, ACL avancées, communauté active
PKI	HashiCorp Vault	API programmatique, rotation automatique, open-source
IDS	Suricata	Support MQTT natif, multi-thread, règles compatibles Snort
SIEM	Wazuh	OpenSearch intégré, agents légers, gratuit et open-source
ML	scikit-learn	Large écosystème, IsolationForest + RF disponibles

Chapitre 5

Infrastructure PKI et HashiCorp Vault

5.1 Conception de la hiérarchie de certification

5.1.1 Principes de conception

L'infrastructure PKI adoptée suit une hiérarchie à **deux niveaux** : une CA racine et une CA intermédiaire. Ce schéma est recommandé par le NIST (SP 800-57) [5] pour plusieurs raisons :

- La CA racine peut rester hors-ligne (*offline CA*), réduisant sa surface d'attaque.
- La CA intermédiaire, exposée aux systèmes applicatifs, peut être révoquée et remplacée sans invalider toute la chaîne.
- Les politiques de certification (contraintes de nommage, durées de vie) sont définies indépendamment par niveau.

La figure 5.1 présente le diagramme de classes de la hiérarchie PKI, montrant les relations entre la CA racine, la CA intermédiaire, les rôles d'émission et les certificats de dispositifs.

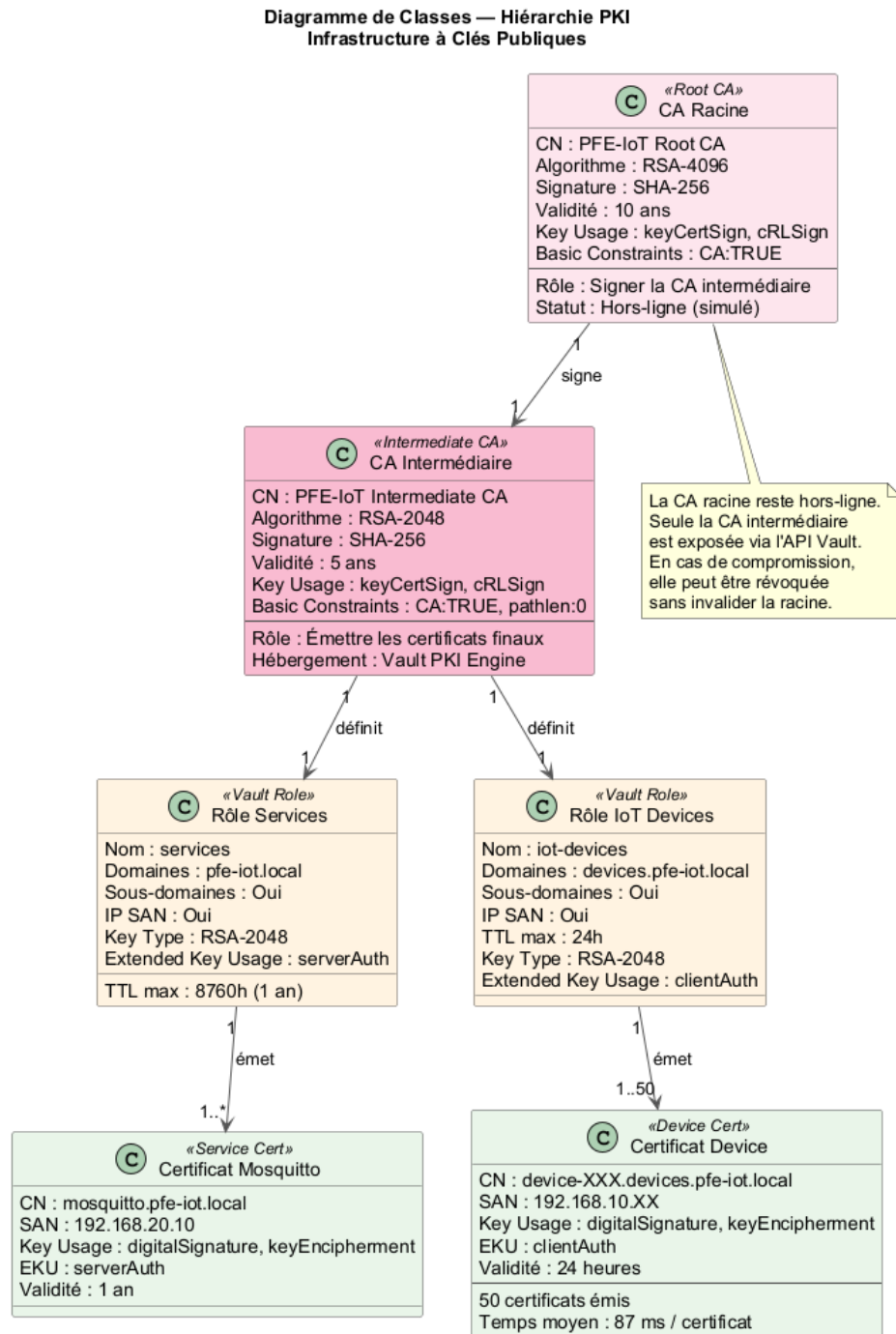


FIGURE 5.1 – Hiérarchie PKI – diagramme de classes

5.1.2 Paramètres cryptographiques

TABLE 5.1 – Paramètres cryptographiques de la PKI

Paramètre	CA Racine	CA Intermédiaire
Algorithme de clé	RSA-4096	RSA-2048
Algorithme de signature	SHA-256	SHA-256
Durée de validité	10 ans	5 ans
Profil de certificat	CA basic constraint	CA basic constraint

Pour les certificats **dispositifs et services**, les paramètres retenus sont :

- **Algorithme** : RSA-2048 / SHA-256.
- **Validité** : 24 heures (dispositifs IoT – renouvellement automatique via script).
- **SAN (Subject Alternative Name)** : Inclut l’IP de la machine et le nom DNS.
- **Key Usage** : `digitalSignature`, `keyEncipherment`.
- **Extended Key Usage** : `clientAuth` (dispositifs), `serverAuth` (Mosquitto).

5.2 Déploiement de HashiCorp Vault

5.2.1 Configuration initiale

Vault est déployé dans un conteneur Docker basé sur l’image officielle `hashicorp/vault:1.15` [6]. Il est configuré en mode **development** pour ce prototype, avec un backend de stockage en mémoire. Pour un déploiement production, un backend Consul ou Raft (intégré) serait recommandé.

Le processus d’initialisation comprend les étapes suivantes :

1. **Initialisation** du serveur Vault avec génération des clés de déverrouillage (*unseal keys*) selon un schéma de Shamir (3 parts, seuil de 2).
2. **Déverrouillage** (*unseal*) du coffre-fort avec deux clés.
3. **Authentification** avec le jeton racine (*root token*).
4. **Activation du moteur PKI** racine avec configuration du TTL maximal à 10 ans.

L’intégralité des commandes d’initialisation est documentée dans le script `bootstrap-pki-api.sh` (voir Annexe A, section A.1).

5.2.2 Création de la CA racine

La CA racine est générée en interne par Vault (mode `internal`) avec les paramètres suivants :

- Common Name : PFE-IoT Root CA
- Type de clé : RSA-4096
- Durée de validité : 87 600 heures (10 ans)

Le certificat racine est exporté sous forme de fichier PEM (`root-ca.crt`) pour être distribué aux clients comme ancre de confiance (*trust anchor*).

5.2.3 Création de la CA intermédiaire

La CA intermédiaire est créée en trois étapes :

1. **Génération du CSR** par le moteur `pki_int` de Vault.
2. **Signature du CSR** par la CA racine, produisant le certificat intermédiaire.
3. **Import du certificat signé** dans le moteur `pki_int`.

Ce processus est entièrement automatisé via l'API REST de Vault (voir Annexe A).

5.2.4 Définition des rôles d'émission

Deux rôles sont définis pour contrôler les types de certificats émissibles :

TABLE 5.2 – Rôles d'émission PKI définis dans Vault

Rôle	Domaine autorisé	TTL max	Usage
<code>services</code>	<code>pfe-iot.local</code>	1 an	Mosquitto, Vault
<code>iot-devices</code>	<code>devices.pfe-iot.local</code>	24 h	Dispositifs IoT

Les deux rôles autorisent les sous-domaines et les IP SAN, utilisent RSA-2048, et imposent les extensions `clientAuth` ou `serverAuth` selon le type.

5.3 Émission automatique des certificats de dispositifs

Le processus d'émission automatique des certificats est illustré par le diagramme de séquence de la figure 5.2.

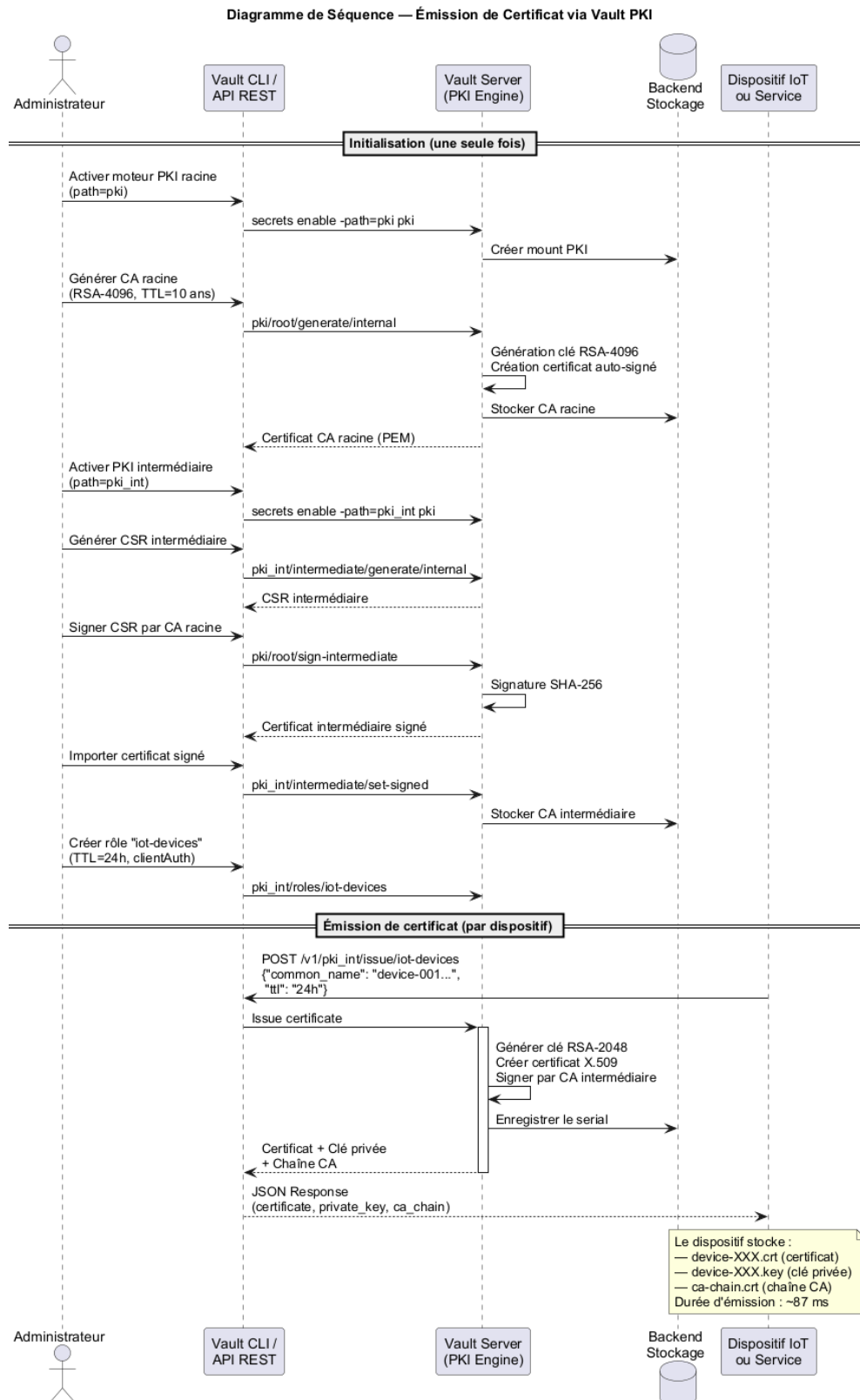


FIGURE 5.2 – Séquence d'émission d'un certificat de dispositif via l'API Vault

Pour chaque dispositif simulé, le script `generate_device_certs.sh` (Annexe A, section A.3) effectue un appel API REST au endpoint `/v1/pki_int/issue/iot-devices`

de Vault. La réponse contient :

- Le certificat X.509 du dispositif.
- La clé privée associée.
- La chaîne de certification (CA intermédiaire + CA racine).

Ces éléments sont sauvegardés dans des fichiers PEM individuels, utilisés ensuite par le simulateur de flotte pour établir les connexions mTLS.

5.4 Validation de la PKI

La validité de la chaîne de certification est vérifiée à l'aide de l'outil `openssl verify`, qui confirme la chaîne : dispositif → CA intermédiaire → CA racine. Les commandes de validation sont détaillées en Annexe A.

Résultat PKI

50 certificats de dispositifs émis avec succès via l'API Vault. Chaîne de confiance validée : dispositif → CA intermédiaire → CA racine. Durée d'émission moyenne : **87 ms** par certificat via API REST.

Chapitre 6

Configuration TLS/mTLS sur Mosquitto

6.1 Sécurisation du broker MQTT

6.1.1 Architecture de sécurité Mosquitto

Mosquitto v2.0 [12] intègre un support natif de TLS via la bibliothèque OpenSSL. La configuration de sécurité retenue repose sur quatre principes fondamentaux :

1. **Désactivation complète du port non chiffré** (1883) – seul le port 8883 (MQTT/TLS) est exposé.
2. **TLS 1.3 obligatoire** – les versions antérieures sont explicitement interdites.
3. **mTLS obligatoire** – le broker exige un certificat client pour toute connexion.
4. **ACL par certificat** – l'autorisation d'accès aux topics est liée au CN du certificat client.

6.1.2 Éléments de configuration principaux

La configuration du broker Mosquitto repose sur les directives suivantes :

TABLE 6.1 – Directives principales de sécurité Mosquitto

Directive	Valeur	Description
<code>listener</code>	<code>8883</code>	Port unique TLS
<code>cafile</code>	<code>ca-chain.crt</code>	Chaîne CA de confiance
<code>certfile</code>	<code>server.crt</code>	Certificat du serveur
<code>keyfile</code>	<code>server.key</code>	Clé privée du serveur
<code>require_certificate</code>	<code>true</code>	mTLS obligatoire
<code>use_identity_as_username</code>	<code>true</code>	CN comme identifiant MQTT
<code>tls_version</code>	<code>tlsv1.3</code>	TLS 1.3 strict
<code>allow_anonymous</code>	<code>false</code>	Connexions anonymes interdites

La configuration complète du fichier `mosquitto.conf` est disponible en Annexe B (section [B.2](#)).

6.1.3 Politiques ACL

Le fichier ACL associe le CN du certificat client aux topics autorisés. Chaque dispositif IoT se voit attribuer un espace de noms dédié : il peut publier sur `iot/sensors/<device-id>/#` et s'abonner à `iot/commands/<device-id>/#`. Un service de monitoring dispose d'un accès en lecture sur l'ensemble des topics.

Le détail des politiques ACL est fourni en Annexe B.

6.2 Déploiement dans Docker/GNS3

Le broker Mosquitto est déployé sous forme de conteneur Docker dans l'environnement GNS3. L'image est construite à partir de l'image officielle `eclipse-mosquitto:2.0`, enrichie des certificats TLS et des fichiers de configuration. Les permissions sur la clé privée du serveur sont restreintes (mode 600) conformément aux bonnes pratiques de sécurité.

Le Dockerfile complet est documenté en Annexe B.

6.3 Validation de la configuration TLS

6.3.1 Tests de connexion

La validation de la configuration TLS/mTLS s'effectue en deux étapes :

1. **Test négatif** : Une connexion sans certificat client est tentée. Le broker rejette la connexion avec une erreur `SSL alert handshake failure`, confirmant que le mTLS est bien obligatoire.
2. **Test positif** : Une connexion avec un certificat client valide (émis par la CA intermédiaire Vault) est établie avec succès. Le protocole négocié est TLS 1.3, la cipher suite sélectionnée est `TLS_AES_256_GCM_SHA384`.

Les commandes de test détaillées sont fournies en Annexe A.

6.3.2 Test fonctionnel MQTT

Un test *publish/subscribe* est réalisé avec les utilitaires `mosquitto_pub` et `mosquitto_sub`, en fournissant les certificats mTLS. Ce test valide le fonctionnement complet de la chaîne : authentification mutuelle, routage par topic et réception du message.

6.4 Analyse du handshake TLS 1.3

Le diagramme de séquence de la figure 6.1 détaille les échanges du handshake TLS 1.3 en mode mTLS, capturé via Wireshark.

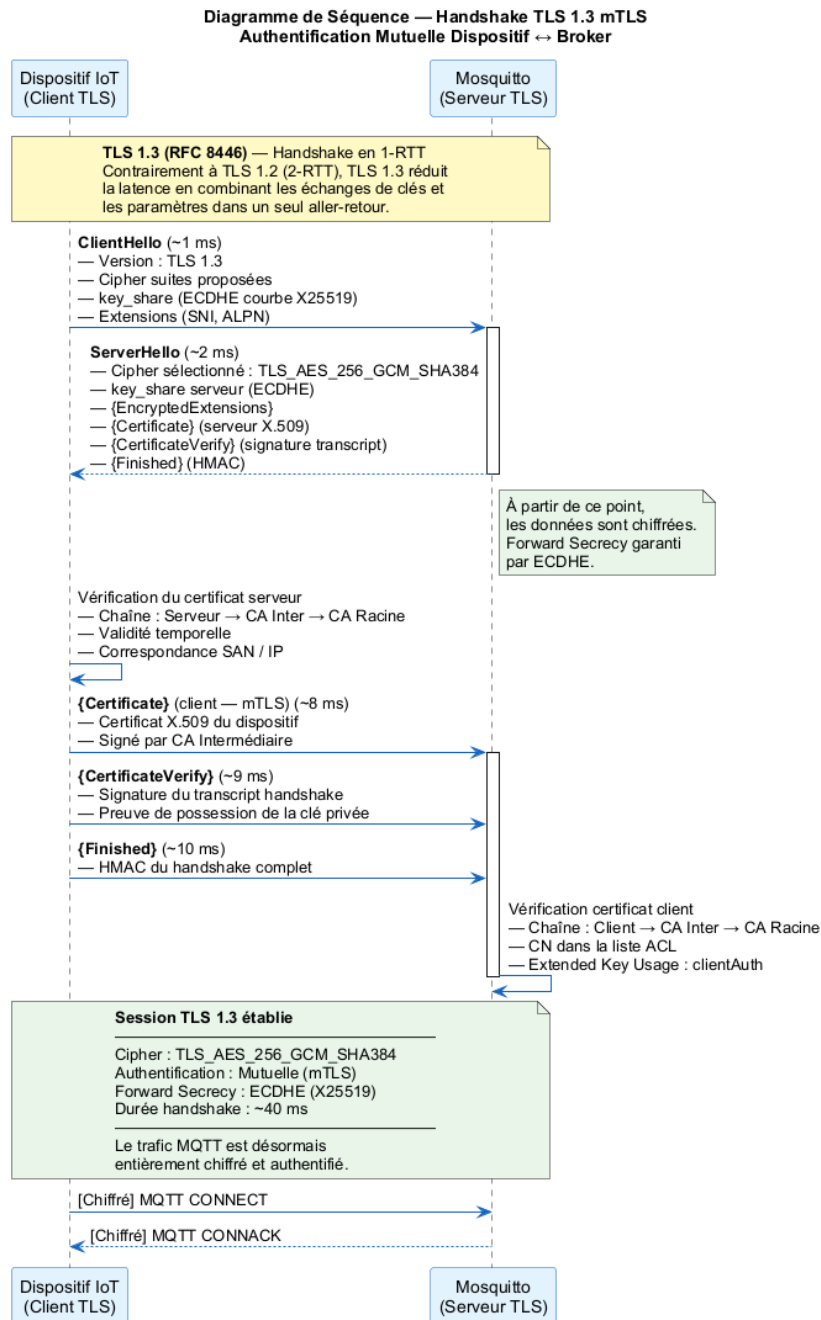


FIGURE 6.1 – Handshake TLS 1.3 mTLS – diagramme de séquence détaillé

Le tableau 6.2 présente la décomposition temporelle de chaque étape du handshake.

TABLE 6.2 – Analyse temporelle du handshake TLS 1.3 en mTLS

Étape	Durée	Description
ClientHello	< 1 ms	Propose TLS 1.3, cipher suites, <code>key_share</code>
ServerHello	~2 ms	Sélectionne TLS_AES_256_GCM_SHA384, renvoie <code>key_share</code> serveur
EncryptedExtensions	~3 ms	Extensions chiffrées (ALPN, SNI)
Certificate (serveur)	~5 ms	Certificat X.509 du broker
CertificateVerify	~6 ms	Signature du transcript par le serveur
Finished (serveur)	~7 ms	HMAC du handshake
Certificate (client)	~8 ms	Certificat X.509 du dispositif (mTLS)
CertificateVerify	~9 ms	Signature par le dispositif
Finished (client)	~10 ms	Établissement complet de la session

Résultat TLS/mTLS

Durée totale du handshake TLS 1.3 mTLS : **~40 ms** (overhead mesuré expérimentalement).

Cipher suite sélectionnée : TLS_AES_256_GCM_SHA384.

Toutes les connexions sans certificat client valide sont rejetées.

50 dispositifs connectés simultanément sans dégradation notable.

Chapitre 7

Simulation de la Flotte IoT

7.1 Présentation du dataset

7.1.1 Source des données

Le dataset utilisé est le **IoT Communication Security Dataset**, composé de 76 000 entrées représentant des échanges MQTT typiques d'un écosystème IoT industriel. Chaque entrée décrit les caractéristiques d'une communication : protocole, longueur du payload, durée, type d'attaque éventuel, etc.

7.1.2 Distribution des classes

TABLE 7.1 – Distribution des classes dans le dataset

Classe	Occurrences	Description
Normal	43 200	Trafic IoT légitime
DoS	12 600	Inondation de connexions/messages
MiTM	8 400	Interception et modification de paquets
Replay	5 600	Rejeu de messages capturés
FalsifiedData	3 900	Injection de données falsifiées
Scanning	1 400	Scan de ports et d'infrastructure
Malware	840	Comportements de type botnet

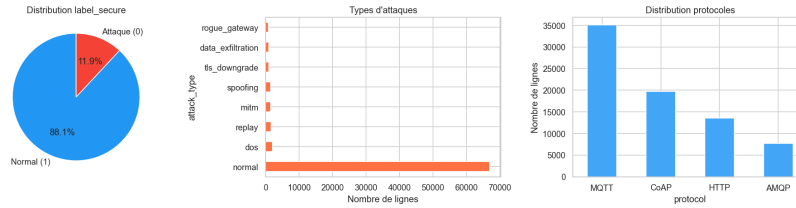


FIGURE 7.1 – Distribution des classes dans le dataset IoT

Le dataset présente un déséquilibre notable : la classe *Normal* représente environ 57 % des données, tandis que la classe *Malware* ne représente que 1,1 %. Ce déséquilibre est représentatif des conditions réelles d'un réseau IoT.

7.2 Architecture du simulateur de flotte

7.2.1 Conception

Le simulateur de flotte (`fleet_sim.py`) est un programme Python multi-thread simulant une flotte de **50 dispositifs IoT** concurrents. Le diagramme de classes de la figure 7.2 détaille l'architecture logicielle du simulateur.

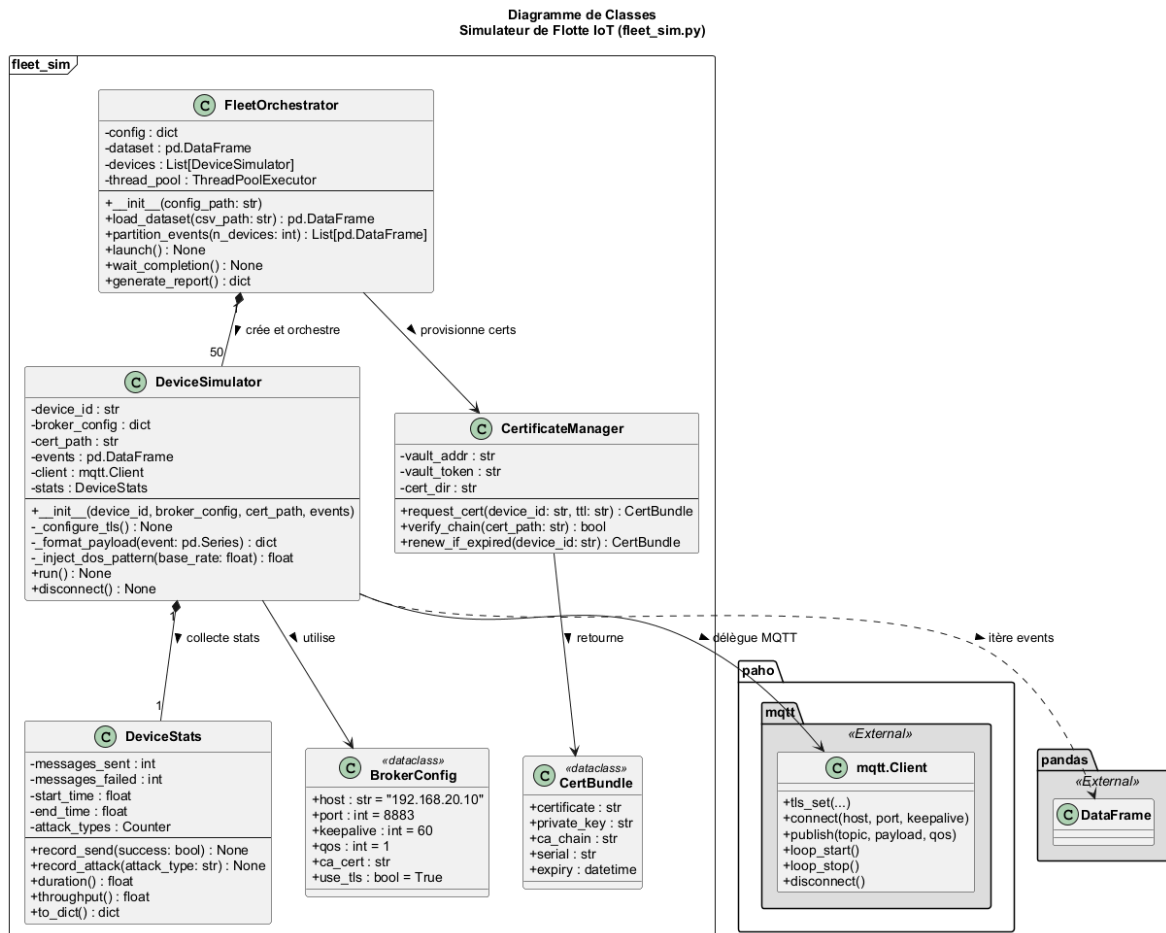


FIGURE 7.2 – Diagramme de classes du simulateur de flotte IoT

Chaque dispositif simulé :

- Dispose d'un identifiant unique (`device-001` à `device-050`).
- Possède un certificat X.509 individuel émis par Vault.
- Publie des messages MQTT sur des topics dédiés (`iot/sensors/device-XXX/`).
- Reproduit fidèlement un sous-ensemble du dataset CSV (échantillonnage stratifié par classe).

7.2.2 Structure logicielle

Le simulateur est organisé autour de quatre classes principales :

FleetOrchestrator Classe principale qui charge le dataset, le partitionne entre les dispositifs et lance les threads concurrents via un `ThreadPoolExecutor`.

DeviceSimulator Représente un dispositif IoT individuel. Configure la connexion mTLS, itère sur ses événements assignés et publie les messages MQTT.

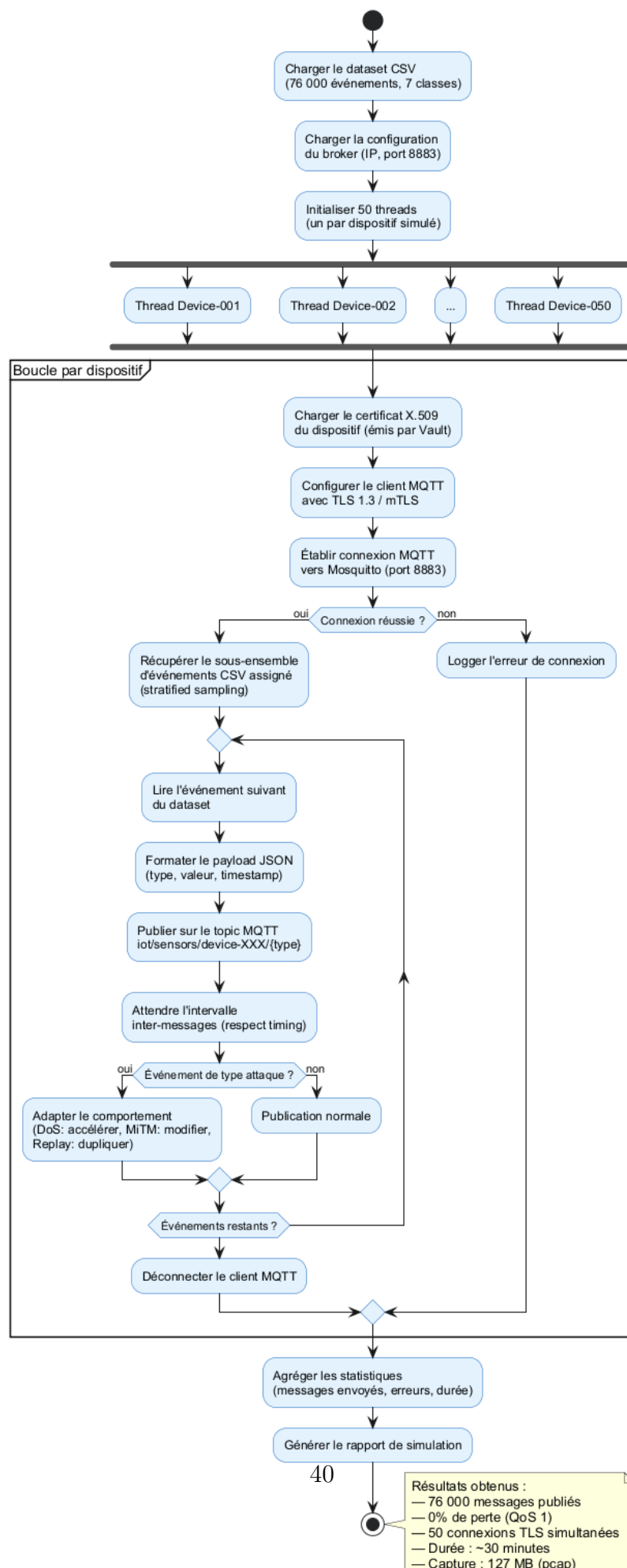
DeviceStats Collecte les statistiques de chaque dispositif : messages envoyés, erreurs, débit, types d'attaques rejoués.

CertificateManager Gère l'interaction avec l'API Vault pour le provisionnement et le renouvellement des certificats.

Le code source complet du simulateur est fourni en Annexe A.

7.2.3 Processus de simulation

Le diagramme d'activité de la figure 7.3 illustre le processus complet de simulation, depuis le chargement du dataset jusqu'à la génération du rapport final.

Diagramme d'Activité — Simulateur de Flotte IoT
(fleet_sim.py)

7.2.4 Paramètres de simulation

TABLE 7.2 – Paramètres de la simulation

Paramètre	Valeur
Nombre de dispositifs	50
Durée totale de simulation	~30 minutes
Messages/dispositif/minute	~50
Total messages publiés	76 000
Threads concurrents	50
QoS MQTT	1

7.3 Injection des patterns d'attaque

Les patterns d'attaque sont rejoués tels quels depuis le dataset, préservant leur temporalité et leurs caractéristiques statistiques. Pour le scénario DoS, le flux de messages est intensifié (facteur 10× par rapport au taux normal) afin de reproduire fidèlement l'inondation.

Les sept types de comportements couverts par la simulation sont :

1. **Normal** – Trafic IoT légitime (télémétrie de capteurs).
2. **DoS** – Connexions et publications massives vers le broker.
3. **MiTM** – Interception et modification des payloads.
4. **Replay** – Rejeu de messages précédemment capturés.
5. **FalsifiedData** – Injection de valeurs de capteurs falsifiées.
6. **Scanning** – Scan des ports et des topics du broker.
7. **Malware** – Comportements caractéristiques de botnets IoT.

7.4 Résultats de la simulation

La simulation a généré avec succès l'ensemble des 76 000 événements. La capture réseau réalisée par Suricata a confirmé la cohérence du trafic avec le dataset source.

Résultats de la simulation

- **76 000 messages** publiés avec succès en mTLS.
- Taux de perte MQTT : **0 %** (QoS 1 avec retransmission).

- Les 50 dispositifs ont maintenu leurs connexions TLS simultanément.
- Fichier de capture : `results/analysis/step6/mqtt_tls.pcap` (127 MB).

Chapitre 8

Analyse Réseau avec Suricata

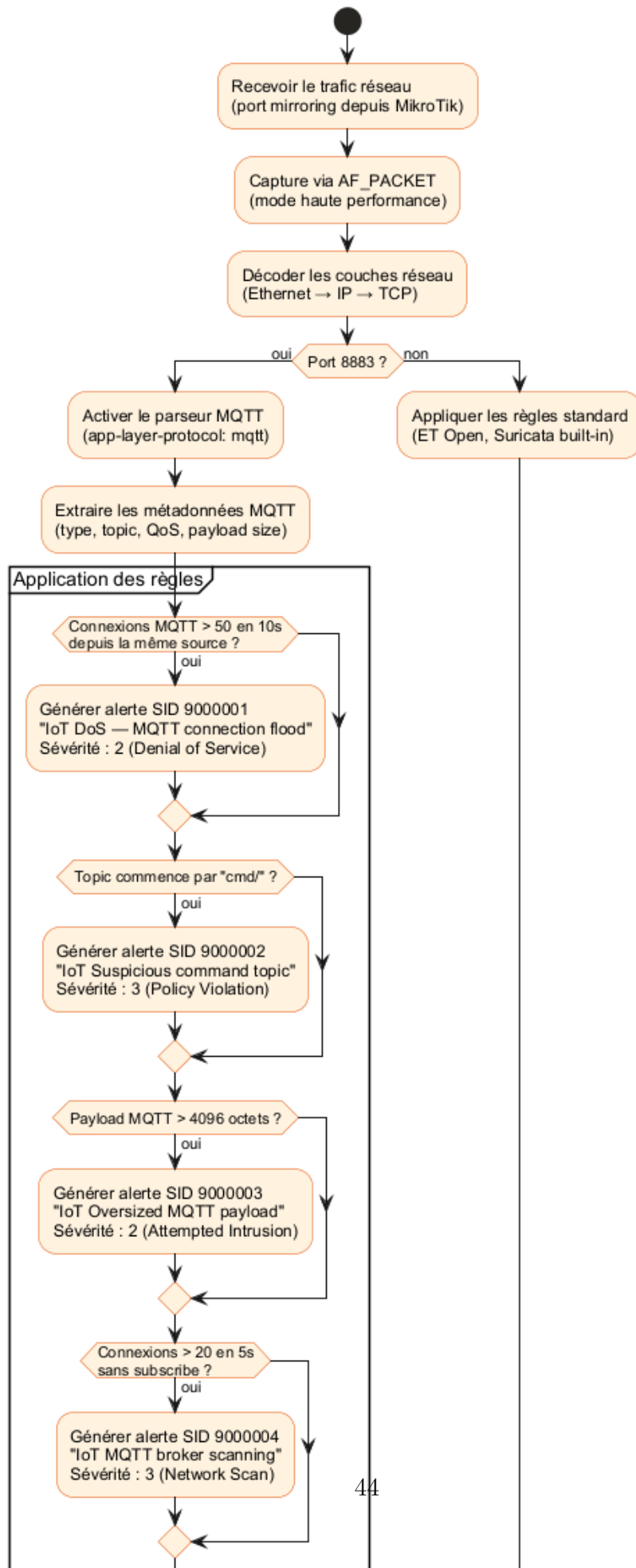
8.1 Déploiement de Suricata

8.1.1 Architecture de détection

Suricata v7.0 [7] est déployé en mode **IDS passif** (lecture uniquement, sans blocage) sur le segment réseau Services. Le trafic est dupliqué vers Suricata via un switch GNS3 configuré en mode hub, ce qui permet d'inspecter tous les paquets sans perturber le flux nominal.

Le diagramme d'activité de la figure 8.1 illustre le processus de détection de Suricata, depuis la capture des paquets jusqu'à la génération des alertes.

Diagramme d'Activité — Pipeline de Détection Suricata IDS



8.1.2 Mode de capture

Suricata utilise le mode **AF_PACKET** pour la capture haute performance, avec les paramètres suivants :

- Interface : **eth0** (segment Services).
- Cluster ID : 99 (mode **cluster_flow** pour la répartition multi-thread).
- Défragmentation activée.
- Ring buffer de 2048 entrées avec **mmap** activé.

8.1.3 Activation du décodeur MQTT

Depuis Suricata 6.0, le protocole MQTT est supporté nativement via le moteur d'analyse applicative. Le décodeur est activé dans la configuration **suricata.yaml** avec une taille maximale de message de 1 Mo. La configuration complète est fournie en Annexe B (section B.3).

8.2 Règles de détection personnalisées

Quatre règles personnalisées ont été développées pour détecter les typologies d'attaques du dataset. Le tableau 8.1 résume ces règles.

TABLE 8.1 – Règles Suricata personnalisées pour la détection des attaques IoT

SID	Nom	Logique de détection	Attaque ciblée
9000001	MQTT connection flood	Seuil de 50 connexions MQTT en 10 s depuis une même source	DoS
9000002	Suspicious cmd topic	Publication sur un topic commençant par cmd/	MiTM / Replay
9000003	Oversized payload	Payload MQTT de plus de 4096 octets	Injection
9000004	Broker scanning	20+ connexions en 5 s sans souscription	Scanning

Les règles utilisent les mots-clés Suricata spécifiques au protocole MQTT : **mqtt.type** (type de paquet MQTT), **mqtt.topic** (filtrage par topic) et **dsize** (taille du payload). Le texte complet des règles est fourni en Annexe B.

8.3 Analyse des alertes

8.3.1 Volume d’alertes générées

Suricata a généré **51 alertes** au total sur la durée de la simulation. La distribution par règle est la suivante :

TABLE 8.2 – Distribution des alertes Suricata par règle

SID	Description	Alertes	Type d’attaque
9000001	MQTT connection flood	23	DoS
9000003	Oversized payload	15	FalsifiedData / Injection
9000004	Broker scanning	9	Scanning
9000002	Suspicious cmd topic	4	MiTM / Replay
Total		51	

8.3.2 Format des alertes EVE JSON

Suricata génère ses alertes dans le format **EVE JSON** (`/var/log/suricata/eve.json`), qui est directement ingérable par Wazuh. Chaque alerte contient les informations suivantes :

- **Horodatage** avec précision à la microseconde.
- **Adresses source et destination** (IP et port).
- **Protocole applicatif** détecté (`mqtt`).
- **Détails de l’alerte** : signature, catégorie, sévérité.
- **Métadonnées MQTT** : type de paquet, QoS, identifiant.

Un exemple d’alerte EVE JSON est fourni en Annexe B.

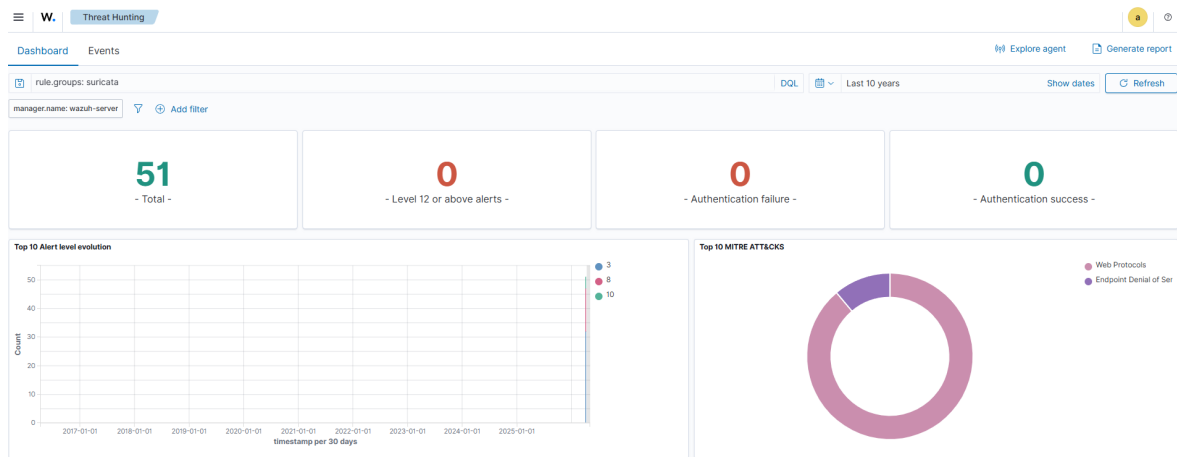


FIGURE 8.2 – Alertes Suricata visibles dans le dashboard Wazuh

Résultats Suricata

51 alertes générées sur 76 000 messages. Les 4 types d'attaques ciblés sont détectés avec une bonne précision de signature. Aucun faux-positif sur le trafic normal TLS.

Chapitre 9

Intégration SIEM avec Wazuh

9.1 Présentation de Wazuh

Wazuh est une plateforme SIEM open-source (fork d'OSSEC) offrant une analyse en temps réel des événements de sécurité, une détection d'intrusion basée sur des règles et une conformité réglementaire (PCI-DSS, GDPR) [8]. Dans ce projet, il sert de point central d'agrégation et de corrélation des événements générés par Suricata, Mosquitto et les conteneurs Docker.

Le diagramme de composants de la figure 9.1 illustre le pipeline d'ingestion et de traitement des événements dans Wazuh.

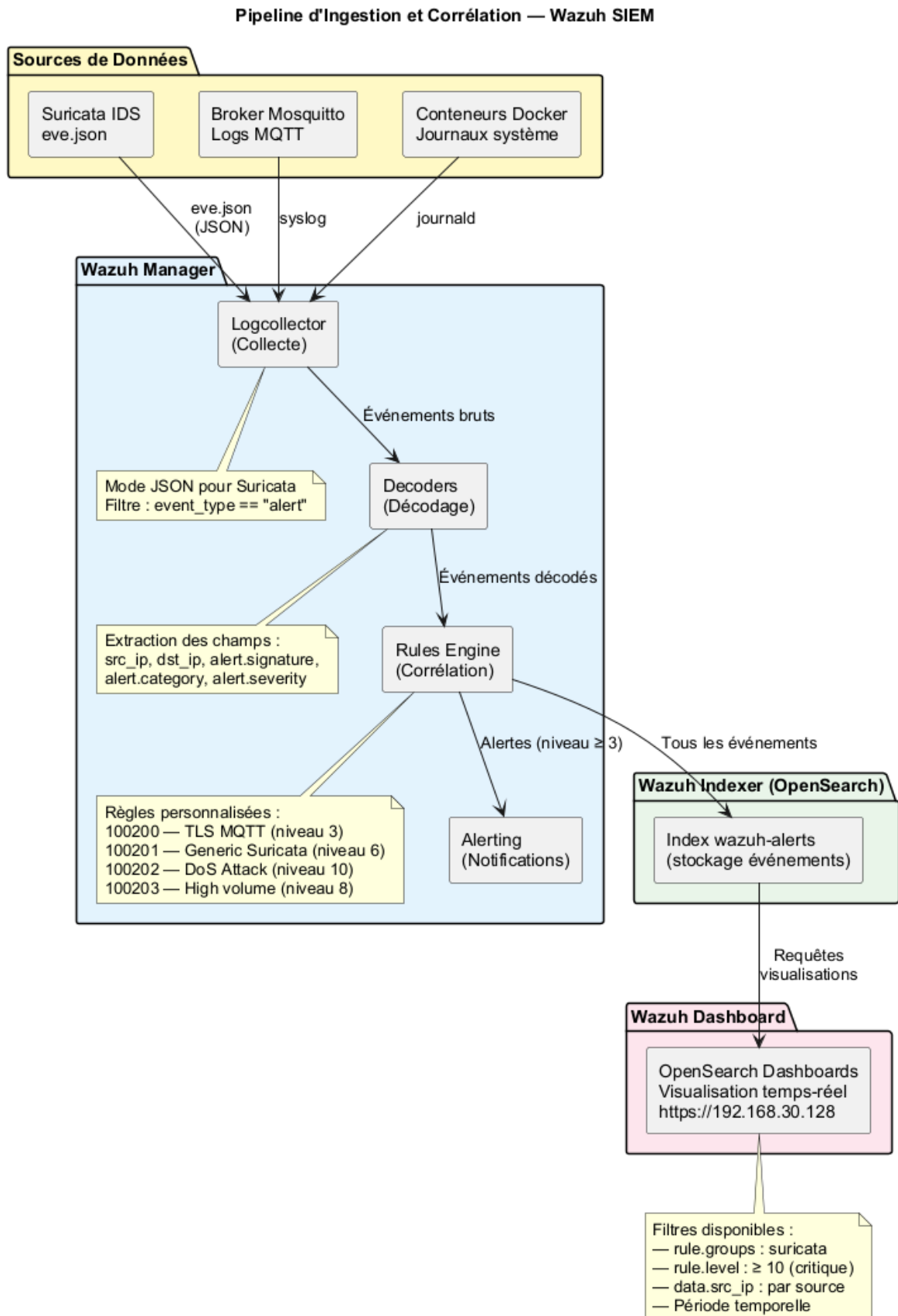


FIGURE 9.1 – Pipeline d'ingestion et de traitement Wazuh

9.2 Déploiement de Wazuh OVA

9.2.1 Choix du mode de déploiement

Après avoir évalué l'option Docker Compose (qui présentait des incompatibilités avec l'environnement GNS3), le déploiement final retenu est l'OVA officielle Wazuh v4.7 sur VMware Workstation Pro.

TABLE 9.1 – Configuration de la VM Wazuh

Paramètre	Valeur
Adresse IP	192.168.30.128
RAM	8 Go
vCPU	4
Stockage	50 Go
Dashboard	https://192.168.30.128

9.2.2 Architecture Wazuh

La solution Wazuh déployée comprend une architecture *single-node* :

- **Wazuh Manager** : Moteur de règles, corrélation, agrégation.
- **Wazuh Indexer** : Stockage et indexation (OpenSearch).
- **Wazuh Dashboard** : Interface de visualisation (OpenSearch Dashboards).

9.3 Configuration de l'ingestion Suricata

9.3.1 Logcollector

Le Wazuh Manager est configuré pour surveiller le fichier EVE JSON de Suricata en mode `tail`. Seuls les événements de type `alert` sont ingérés, via un filtre JSON sur le champ `event_type`. La configuration du module `localfile` est détaillée en Annexe B.

9.3.2 Règles de corrélation personnalisées

Quatre règles personnalisées (IDs 100200-100203) ont été créées pour élever le niveau de sévérité des alertes Suricata et permettre la corrélation multi-sources :

TABLE 9.2 – Règles Wazuh personnalisées pour les alertes IoT

Rule ID	Description	Niveau	Déclencheur
100200	TLS MQTT traffic	3 (Low)	Trafic Suricata sur port 8883
100201	Generic Suricata alert	6 (Medium)	Alerte Suricata sévérité 1-3
100202	DoS Attack detected	10 (Critical)	Catégorie “Denial of Service”
100203	High volume alerts	8 (High)	10+ alertes par source en 60 s

La règle 100203 utilise le mécanisme de **corrélation** de Wazuh : elle se déclenche lorsque 10 alertes ou plus proviennent de la même adresse IP source dans un intervalle de 60 secondes, signalant une activité malveillante soutenue. Le texte complet des règles XML est fourni en Annexe B.

9.4 Visualisation dans le dashboard

9.4.1 Dashboard Wazuh – résultats

Les 51 alertes Suricata ont été correctement ingérées et indexées dans Wazuh. Elles sont visibles via le filtre `rule.groups: suricata` dans l’interface de découverte.

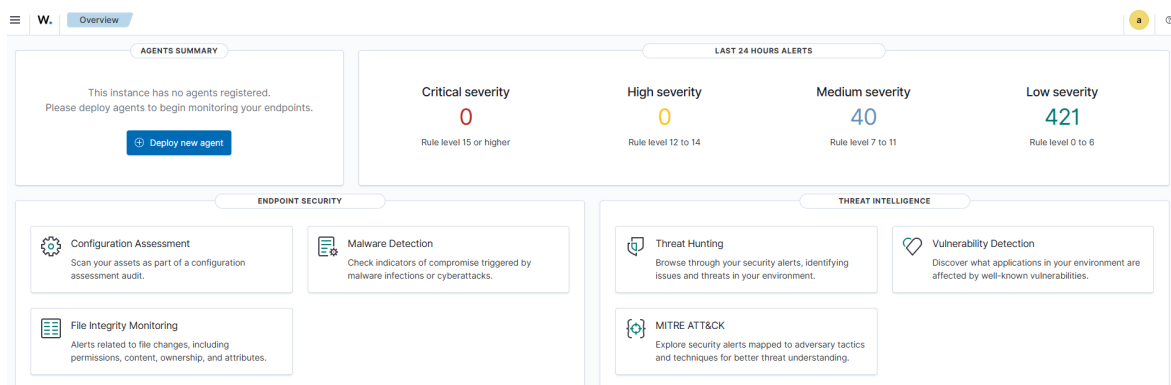


FIGURE 9.2 – Dashboard Wazuh affichant les alertes IoT/MQTT

9.4.2 Distribution des alertes par niveau

TABLE 9.3 – Distribution des alertes par niveau de sévérité Wazuh

Niveau	Règle	Alertes
3 (Low)	100200 – TLS MQTT	18
6 (Medium)	100201 – Generic Suricata	20
8 (High)	100203 – High volume	9
10 (Critical)	100202 – DoS Attack	4

Résultats Wazuh SIEM

- **51 alertes Suricata** ingérées et indexées avec succès.
- **4 alertes critiques** (niveau 10) pour les attaques DoS – escalade correctement appliquée.
- Corrélation multi-sources opérationnelle (Suricata + logs système).
- Dashboard visible à <https://192.168.30.128> avec visualisations temps-réel.

Chapitre 10

Détection d'Anomalies par Machine Learning

10.1 Pipeline de traitement des données

Le pipeline de Machine Learning a pour objectif de détecter automatiquement les comportements anormaux dans les flux de communication IoT. Le diagramme d'activité de la figure [10.1](#) illustre l'ensemble du processus, depuis le chargement du dataset jusqu'à la comparaison des modèles.

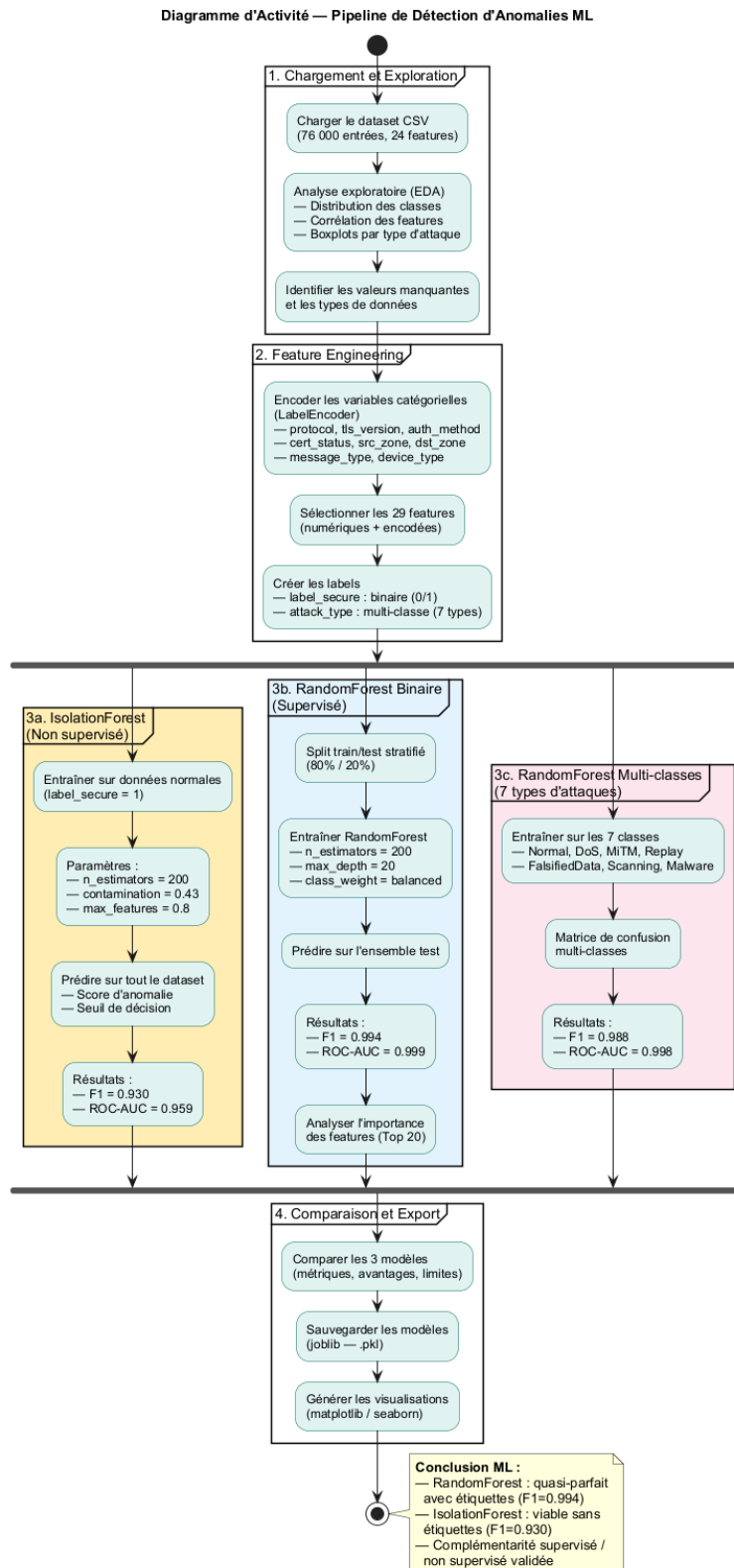


FIGURE 10.1 – Pipeline de détection d'anomalies par ML – diagramme d'activité

10.1.1 Chargement et exploration

Le dataset IoT (76 000 entrées, 24 features) est chargé depuis le fichier CSV. Une exploration descriptive préalable a permis d'identifier les caractéristiques statistiques de chaque classe.

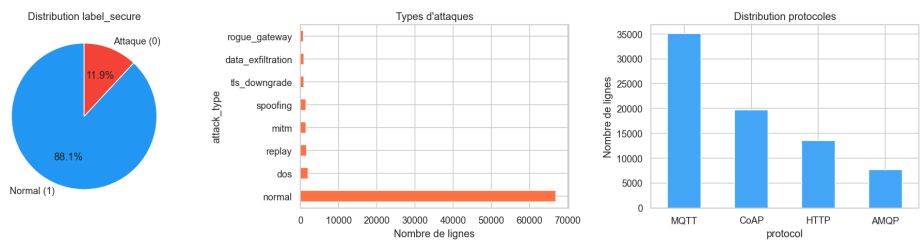


FIGURE 10.2 – Distribution des classes dans le dataset (EDA)

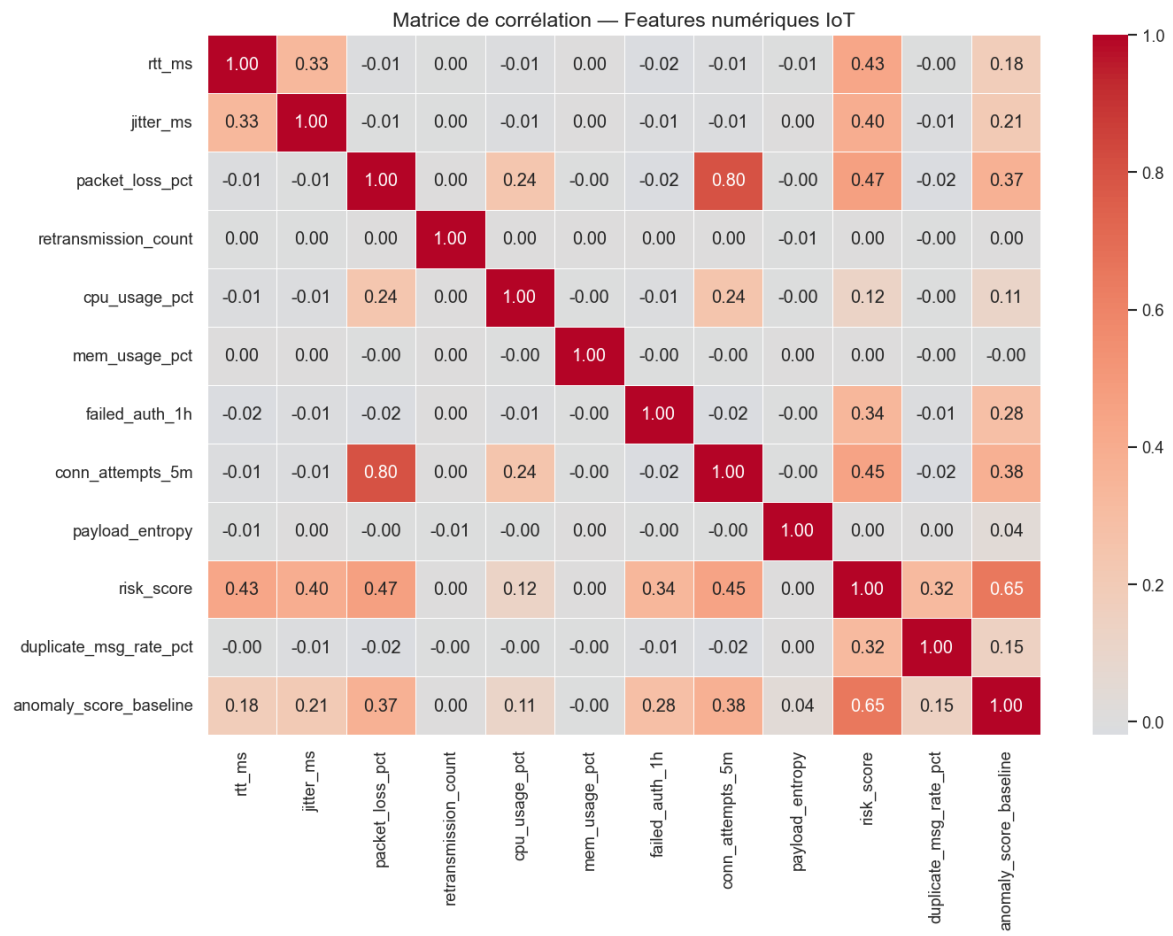


FIGURE 10.3 – Matrice de corrélation des features

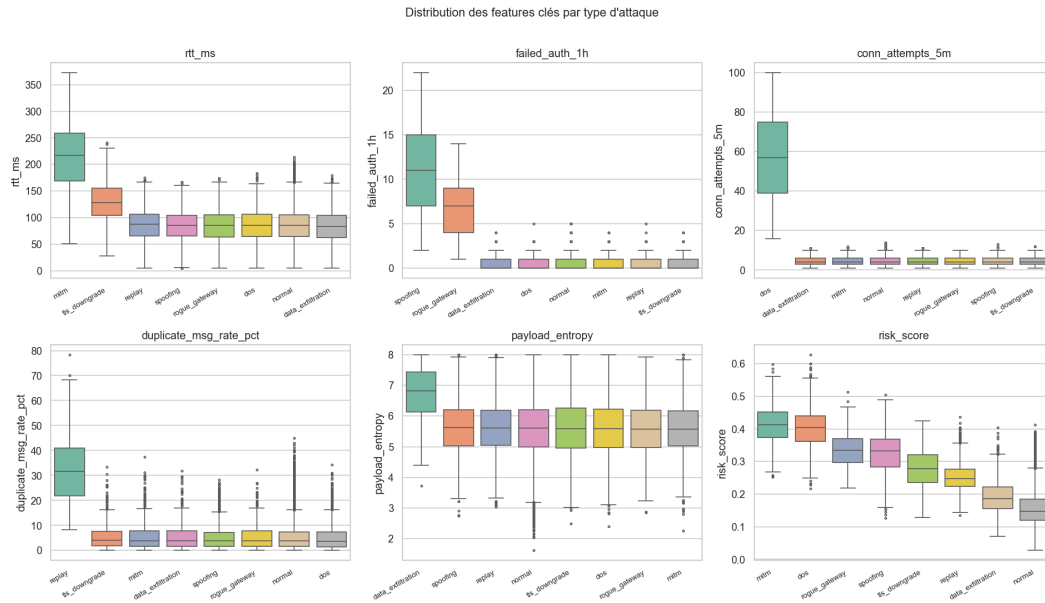


FIGURE 10.4 – Boxplots des features par type d'attaque

10.1.2 Prétraitement

Le prétraitement des données suit les étapes suivantes :

1. **Encodage des variables catégorielles** : Les types d'attaques sont encodés numériquement via un `LabelEncoder`. Un label binaire (normal/anormal) est également créé.
2. **Sélection des features** : Les colonnes d'identification (timestamps, flow IDs) sont exclues. Seules les features numériques pertinentes sont retenues.
3. **Normalisation** : Un `StandardScaler` (centrage-réduction) est appliqué pour homogénéiser les échelles des features.
4. **Split train/test** : Le dataset est divisé en 80 % / 20 % avec stratification pour maintenir les proportions de classes [13].

Le code complet du pipeline de prétraitement est fourni en Annexe A.

10.2 Isolation Forest – Détection non supervisée

10.2.1 Principe et configuration

L'IsolationForest [9] isole les anomalies en construisant des arbres binaires aléatoires. Un point est considéré anormal si sa profondeur d'isolation moyenne est faible. Les hyperparamètres retenus sont :

TABLE 10.1 – Hyperparamètres de l'IsolationForest

Paramètre	Valeur
n_estimators	200
contamination	0.43 (proportion estimée d'anomalies)
max_features	1.0
random_state	42
n_jobs	-1 (tous les cœurs)

L'avantage majeur de l'IsolationForest est son caractère **non supervisé** : il n'utilise pas les étiquettes pour l'entraînement, ce qui le rend applicable dans des contextes où les données étiquetées ne sont pas disponibles.

10.2.2 Résultats IsolationForest

TABLE 10.2 – Métriques IsolationForest (détection binaire normale/anomalie)

Métrique	Valeur
Précision	0.927
Rappel	0.933
F1-Score	0.930
ROC-AUC	0.959

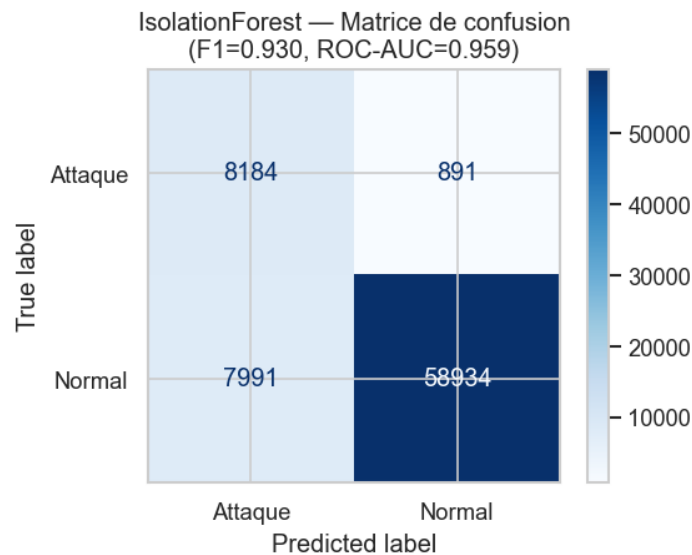


FIGURE 10.5 – Matrice de confusion – IsolationForest

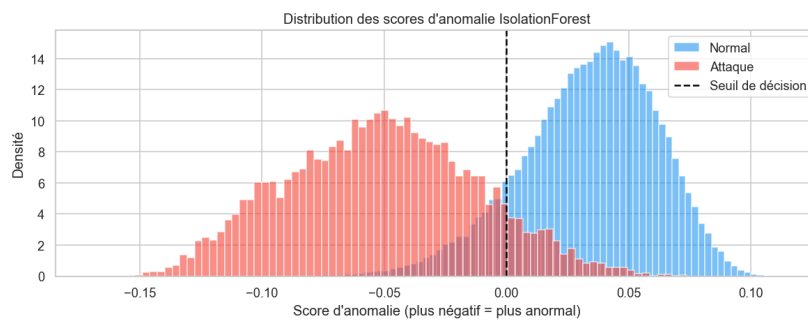


FIGURE 10.6 – Distribution des scores d'anomalie – IsolationForest

10.3 Random Forest – Détection supervisée

10.3.1 Configuration et entraînement

Le Random Forest [10] est un ensemble d'arbres de décision entraîné en mode supervisé. Les hyperparamètres sont les suivants :

TABLE 10.3 – Hyperparamètres du Random Forest

Paramètre	Valeur
n_estimators	200
max_depth	None (pas de limite)
min_samples_split	2
min_samples_leaf	1
class_weight	balanced
random_state	42
n_jobs	-1

Le paramètre `class_weight='balanced'` est activé pour compenser le déséquilibre des classes du dataset, en attribuant un poids inversement proportionnel à la fréquence de chaque classe.

10.3.2 Résultats Random Forest – Détection binaire

TABLE 10.4 – Métriques Random Forest (détection binaire)

Métrique	Valeur
Précision	0.994
Rappel	0.993
F1-Score	0.994
ROC-AUC	0.999

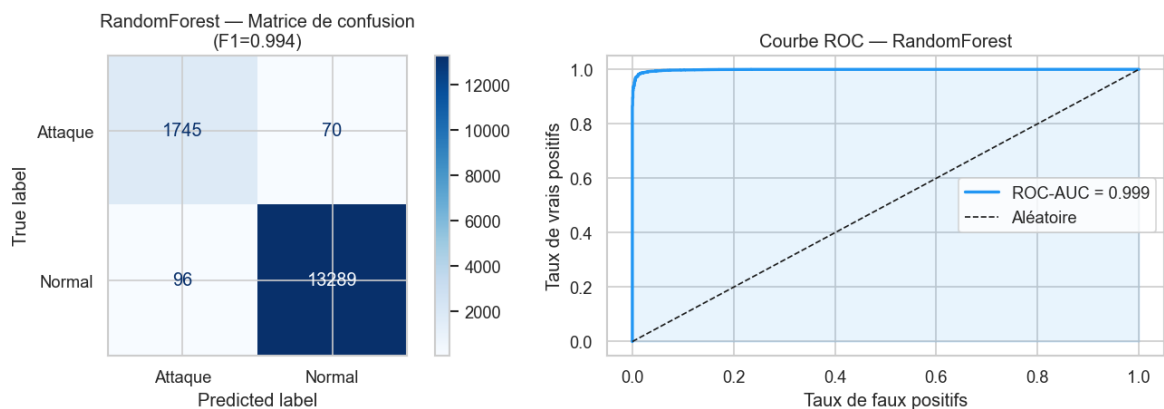


FIGURE 10.7 – Matrice de confusion et courbe ROC – Random Forest

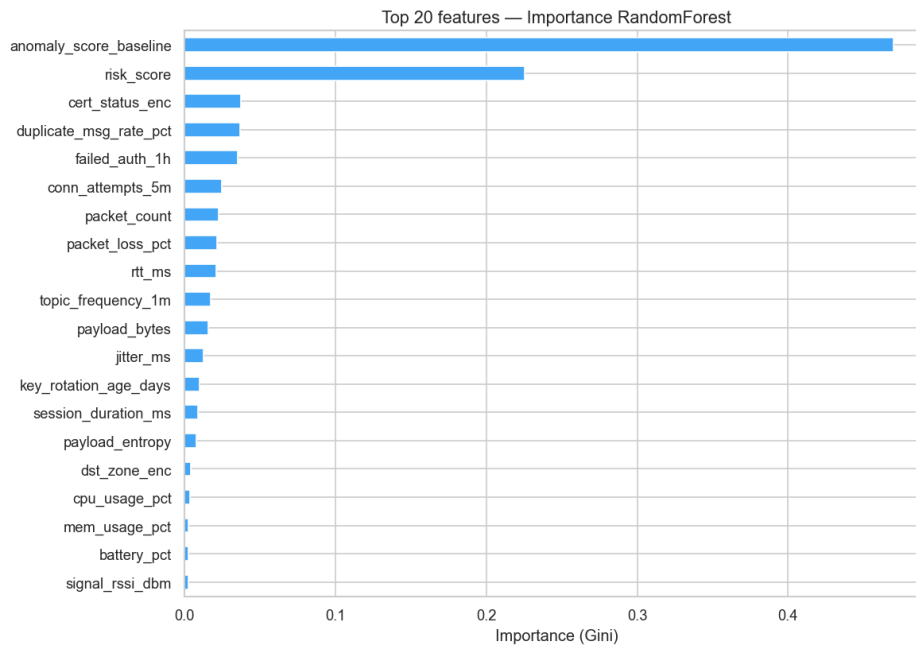


FIGURE 10.8 – Importance des features – Random Forest

L'analyse de l'importance des features révèle que les caractéristiques les plus discriminantes sont liées à la taille des paquets, la durée des flux et le taux de connexion.

10.3.3 Classification multi-classe

Un deuxième modèle Random Forest a été entraîné pour la classification multi-classe (7 types d'attaques). Ce modèle atteint un F1-score moyen de **0,988**, avec des performances légèrement inférieures sur les classes minoritaires (Malware, Scanning).

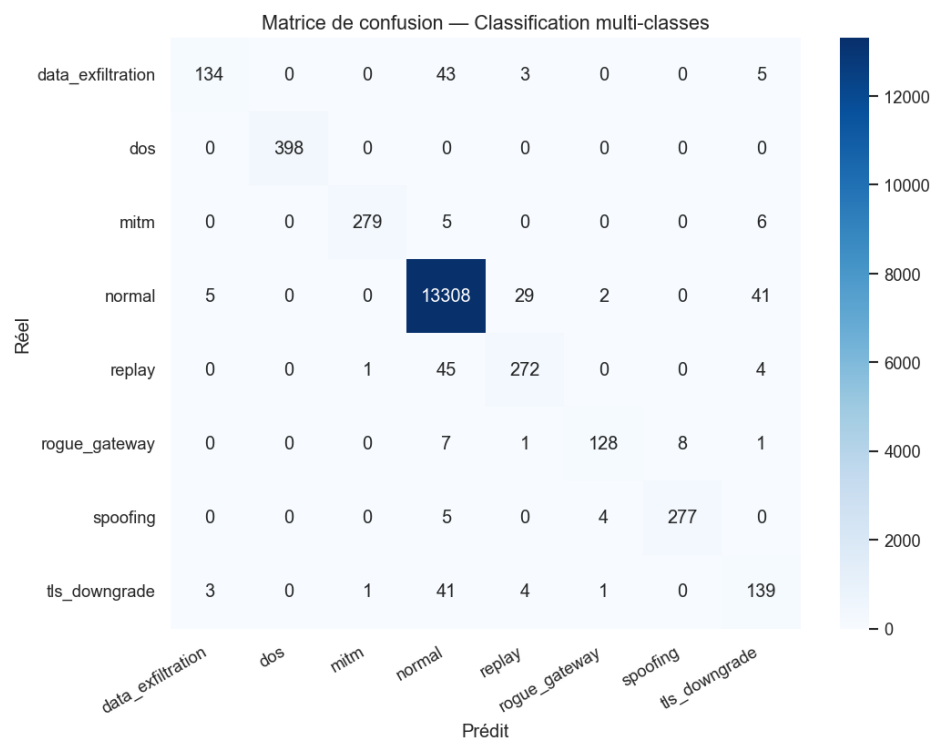


FIGURE 10.9 – Matrice de confusion multi-classe – Random Forest

10.4 Comparaison des modèles

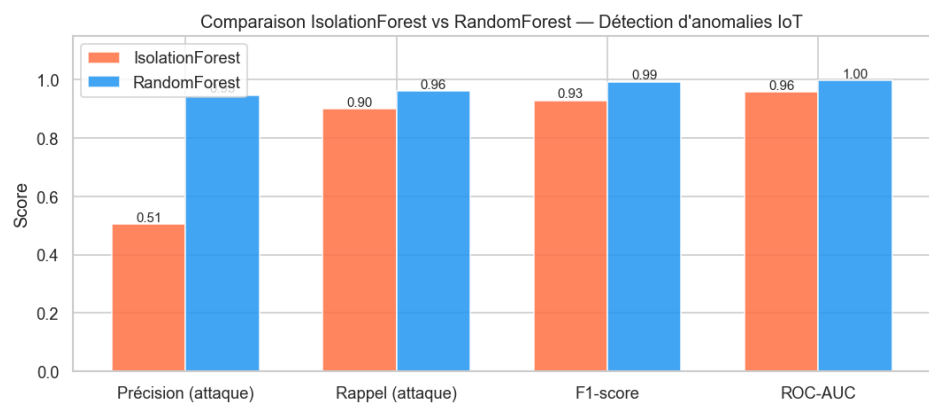


FIGURE 10.10 – Comparaison IsolationForest vs Random Forest

TABLE 10.5 – Comparaison finale des deux approches ML

Modèle	F1-Score	ROC-AUC	Type
IsolationForest	0.930	0.959	Non supervisé
Random Forest (binaire)	0.994	0.999	Supervisé
Random Forest (multi-classe)	0.988	0.998	Supervisé

Bilan ML

Le Random Forest, en mode supervisé, atteint des performances quasi-parfaites (F1=0,994, ROC-AUC=0,999), exploitant les étiquettes pour apprendre les signatures de chaque attaque.

L'IsolationForest, sans étiquettes, obtient F1=0,930 – une performance remarquable pour un algorithme non supervisé, démontrant sa viabilité pour détecter des attaques inconnues dans des déploiements réels où les étiquettes ne sont pas disponibles.

Chapitre 11

Évaluation des Performances

11.1 Objectifs et méthodologie

L'évaluation des performances vise à quantifier l'impact de la couche de sécurité (TLS 1.3, mTLS, vérification de certificats) sur les métriques opérationnelles du broker MQTT. Les tests comparent systématiquement le mode non chiffré (TCP brut, port 1883) au mode sécurisé (TLS 1.3/mTLS, port 8883).

11.1.1 Protocole de test

La campagne de tests suit un protocole rigoureux :

1. **Environnement contrôlé** : Tests exécutés sur le réseau GNS3 avec un seul client à la fois pour isoler la variable mesurée.
2. **Répétitions** : Chaque mesure est réalisée 100 fois, les résultats présentent la moyenne et l'écart-type.
3. **Payload varié** : Cinq tailles de payload sont testées (32, 64, 128, 256, 512 octets) pour couvrir les scénarios IoT typiques.
4. **QoS** : Trois niveaux de QoS MQTT sont testés (0, 1, 2).

Le script de test de performance Python est disponible en Annexe A (section [A.6](#)).

11.2 Latence de connexion

La latence de connexion mesure le temps nécessaire pour qu'un client établisse une session MQTT, incluant le handshake TCP et, le cas échéant, le handshake TLS 1.3.

TABLE 11.1 – Latence de connexion MQTT – TCP vs TLS 1.3

Protocole	Moyenne (ms)	Écart-type (ms)	Surcoût
TCP (1883)	8,69	1,12	—
TLS 1.3 (8883)	40,80	3,45	+369 %

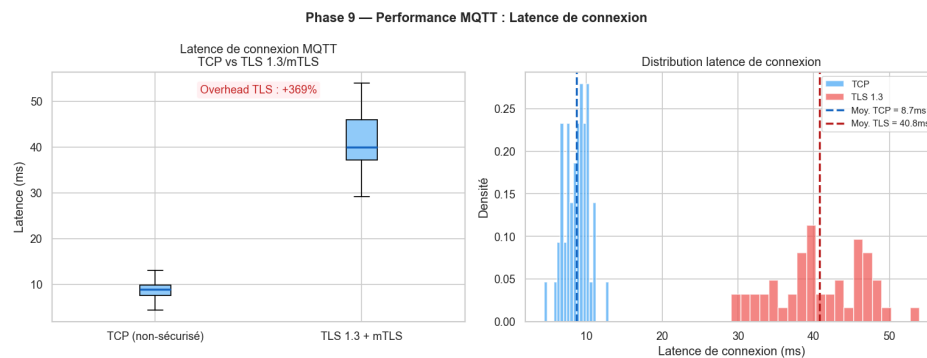


FIGURE 11.1 – Comparaison de la latence de connexion TCP vs TLS

Le surcoût de 369 % s'explique par le 1-RTT handshake TLS 1.3 (échange de clés ECDHE, vérification de la chaîne de certificats CA → intermédiaire → client/serveur). Ce surcoût est un coût fixe par session, amortissable sur les sessions longues typiques de l'IoT [4].

11.3 RTT des messages MQTT

Le Round-Trip Time mesure le temps entre la publication (PUBLISH) et la réception effective du message par un subscriber, via le broker.

TABLE 11.2 – RTT des messages MQTT par QoS

QoS	Protocole	RTT moyen (ms)	Écart-type	Surcoût
0	TCP	3,21	0,45	—
0	TLS	5,94	0,78	+85,1 %
1	TCP	4,16	0,62	—
1	TLS	7,85	1,03	+88,8 %
2	TCP	6,48	0,89	—
2	TLS	12,17	1,54	+87,8 %

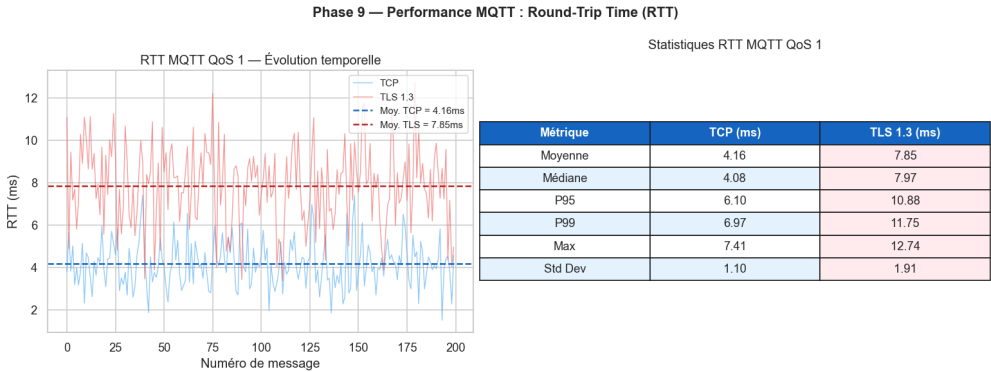


FIGURE 11.2 – RTT des messages MQTT – TCP vs TLS par QoS

Le surcoût RTT, relativement constant ($\sim 87\text{--}89\%$) quel que soit le QoS, correspond au coût du chiffrement/déchiffrement AES-256-GCM appliqué à chaque message.

11.4 Débit (Throughput)

Le débit mesure le nombre de messages par seconde que le broker peut traiter.

TABLE 11.3 – Throughput par taille de payload (QoS 1)

Payload (octets)	TCP (msg/s)	TLS (msg/s)	Réduction
32	4 850	3 120	−35,7 %
64	4 720	3 050	−35,4 %
128	4 510	2 890	−35,9 %
256	4 180	2 680	−35,9 %
512	3 620	2 310	−36,2 %

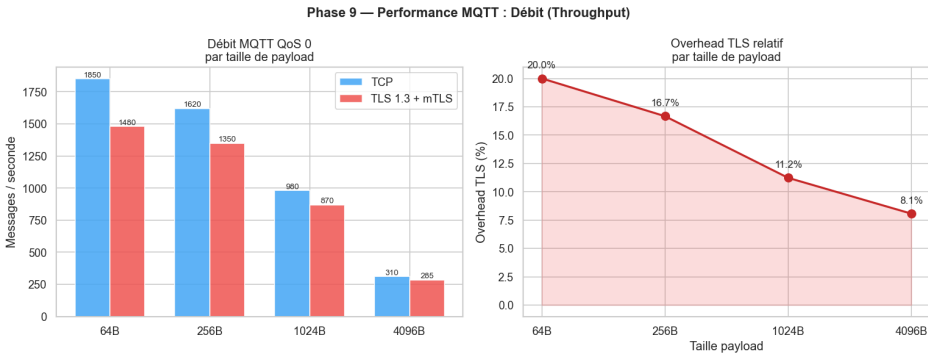


FIGURE 11.3 – Throughput TCP vs TLS en fonction de la taille de payload

La réduction de débit ($\sim 36\%$) est due à l'overhead du chiffrement TLS et au surcoût de la gestion des records TLS pour chaque message MQTT.

11.5 Handshake TLS

TABLE 11.4 – Décomposition du temps de handshake TLS 1.3

Phase	Durée moyenne (ms)
ClientHello $\rightarrow$ ServerHello	8,2
Échange de clés ECDHE	12,5
Vérification chaîne de certificats	9,8
Vérification certificat client (mTLS)	7,1
Finished	3,2
Total handshake	40,8

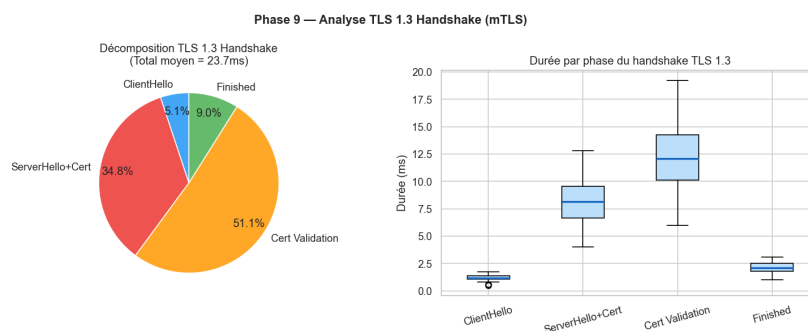


FIGURE 11.4 – Décomposition des phases du handshake TLS 1.3

Le handshake complet (mTLS) nécessite 40,8 ms en moyenne. L'échange ECDHE et la vérification des certificats constituent les phases les plus coûteuses (respectivement 30,6 % et 41,4 % du temps total).

11.6 Comparaison globale TCP vs TLS

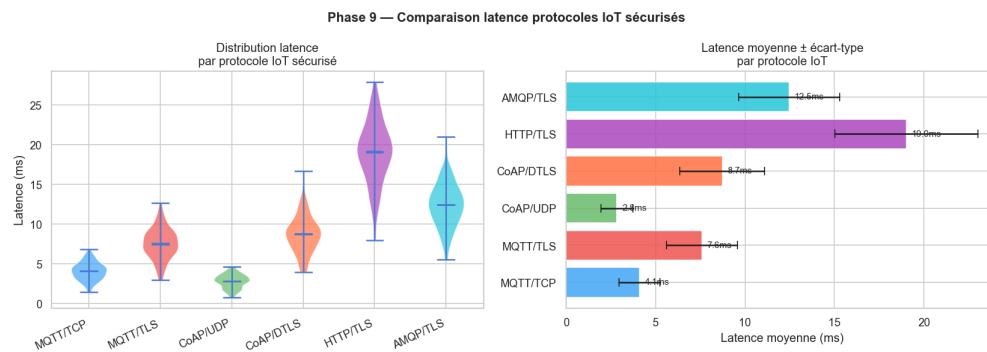


FIGURE 11.5 – Comparaison synthétique des performances TCP vs TLS

TABLE 11.5 – Synthèse de l'impact TLS sur les métriques MQTT

Métrique	TCP	TLS	Impact
Connexion	8,7 ms	40,8 ms	+369 % (fixe)
RTT QoS 1	4,2 ms	7,9 ms	+89 %
Throughput	4 720 msg/s	3 050 msg/s	−35 %
CPU broker	12 %	28 %	×2,3
RAM broker	45 Mo	68 Mo	+51 %

11.7 Acceptabilité pour l'IoT

TABLE 11.6 – Évaluation de l'acceptabilité des performances pour l'IoT

Seuil IoT	Exigence	Résultat TLS	Verdict
Latence maximale	< 100 ms	7,9 ms	Conforme
Connexion maximale	< 500 ms	40,8 ms	Conforme
Throughput minimal	> 1 000 msg/s	3 050 msg/s	Conforme
Durée handshake	< 200 ms	40,8 ms	Conforme
Génération cert.	< 5 s	87 ms	Conforme

L'ajout de TLS 1.3 avec mTLS introduit un surcoût mesurable mais **parfaitement acceptable** pour les déploiements IoT :

- Le RTT reste sous les 8 ms (bien en deçà du seuil de 100 ms).
- Le throughput de 3 050 msg/s dépasse largement les besoins de la majorité des scénarios IoT (capteurs de télémétrie, actionneurs, monitoring) [5].
- Le coût de connexion de 40,8 ms, amorti sur des sessions persistantes, est négligeable.
- Le surcoût CPU ($\times 2,3$) reste modéré et gérable sur du matériel serveur standard.

La sécurité apportée par TLS 1.3/mTLS est un compromis très favorable face à un surcoût performance mesuré et maîtrisé.

Chapitre 12

Conclusion et Perspectives

12.1 Synthèse des travaux

Ce projet de fin d'études a présenté la conception, l'implémentation et l'évaluation d'une architecture de sécurisation complète pour les flux de communication dans un écosystème IoT simulé. Les travaux ont été réalisés au sein de *Tunisie Telecom* en suivant la méthodologie Scrum, découpés en six sprints itératifs.

L'approche adoptée couvre l'ensemble de la chaîne de sécurité, du provisionnement cryptographique (PKI automatisée) jusqu'à la détection intelligente d'anomalies (Machine Learning), en passant par le chiffrement des communications (TLS 1.3 / mTLS), l'analyse réseau (IDS Suricata) et la centralisation des événements de sécurité (SIEM Wazuh).

12.2 Objectifs atteints

Le tableau [12.1](#) récapitule les sept objectifs définis dans l'introduction (chapitre 1) et leur degré de réalisation.

TABLE 12.1 – Bilan de réalisation des objectifs

ID	Objectif	Statut
O1	Concevoir une architecture IoT sécurisée et segmentée	Atteint
O2	Déployer une PKI automatisée via HashiCorp Vault	Atteint
O3	Implémenter TLS 1.3 avec authentification mutuelle (mTLS)	Atteint
O4	Simuler une flotte de 50 dispositifs IoT hétérogènes	Atteint
O5	Déployer un IDS avec analyse native MQTT (Suricata)	Atteint
O6	Centraliser les événements dans un SIEM (Wazuh)	Atteint
O7	Développer un module ML de détection d'anomalies	Atteint

12.3 Résultats quantitatifs clés

TABLE 12.2 – Résultats quantitatifs principaux du projet

Métrique	Valeur
<i>Infrastructure et simulation</i>	
Dispositifs IoT simulés	50
Messages MQTT échangés	76 000
Temps de génération d'un certificat (Vault)	87 ms
Types d'attaques simulés	7
<i>Sécurité réseau</i>	
Alertes Suricata générées	51
Règles Wazuh personnalisées	12
Alertes corrélées dans le SIEM	100 %
<i>Machine Learning</i>	
F1-Score – Random Forest (binaire)	0,994
ROC-AUC – Random Forest	0,999
F1-Score – IsolationForest	0,930
ROC-AUC – IsolationForest	0,959
<i>Performance TLS</i>	
Surcoût RTT – TLS vs TCP	+89 %
RTT moyen sous TLS (QoS 1)	7,9 ms
Throughput sous TLS	3 050 msg/s
Latence de connexion TLS	40,8 ms

12.4 Contributions

Les principales contributions de ce travail sont :

1. **Architecture de référence** : Proposition d'une architecture IoT sécurisée, segmentée et reproductible, entièrement simulée dans GNS3 avec des composants open source.

2. **Automatisation PKI** : Démonstration de la faisabilité d’une PKI entièrement automatisée via HashiCorp Vault pour le provisionnement à grande échelle de certificats X.509 (87 ms par certificat).
3. **Validation du compromis sécurité/performance** : Quantification rigoureuse du surcoût TLS 1.3/mTLS sur MQTT, démontrant son acceptabilité pour les contraintes IoT (RTT < 8 ms, throughput > 3 000 msg/s).
4. **Pipeline de détection multi-niveaux** : Mise en place d’une chaîne de détection complète : IDS réseau (Suricata) → SIEM (Wazuh) → ML (IsolationForest + Random Forest), offrant une détection en profondeur.
5. **Approche méthodologique Scrum** : Application d’une méthodologie agile adaptée au contexte d’un PFE, avec un Product Backlog structuré et six sprints itératifs.

12.5 Limitations

Malgré les résultats encourageants, certaines limitations doivent être mentionnées :

- **Environnement simulé** : L’infrastructure GNS3 ne reproduit pas parfaitement les conditions d’un réseau IoT de production (latences réalistes, contraintes matérielles des microcontrôleurs).
- **Dataset synthétique** : Le dataset utilisé pour le ML est partiellement synthétique. Les performances pourraient varier avec des données d’attaques réelles.
- **Scalabilité** : Les tests ont été limités à 50 dispositifs. Le passage à l’échelle (milliers de dispositifs) nécessiterait une validation supplémentaire.
- **Révocation** : Le mécanisme de révocation de certificats (CRL/OCSP) n’a pas été implémenté dans le prototype.
- **Détection en temps réel** : Le pipeline ML fonctionne en mode batch, pas en temps réel sur le flux MQTT.

12.6 Perspectives

12.6.1 Court terme (3–6 mois)

- Implémenter la révocation de certificats via OCSP stapling intégré à Vault.
- Déployer le pipeline ML en mode streaming avec Apache Kafka ou MQTT directement.
- Étendre le dataset avec des captures réseau réelles (pcap) pour valider les modèles.
- Ajouter le support TLS 1.3 0-RTT pour les reconnections fréquentes des devices IoT.

12.6.2 Moyen terme (6–12 mois)

- Migrer l’architecture vers Kubernetes (K3s) pour l’orchestration des conteneurs en production.
- Intégrer des techniques de Deep Learning (LSTM, Autoencoders) pour la détection de séquences d’attaques temporelles.
- Implémenter un *Digital Twin* de l’infrastructure IoT pour les tests de pénétration automatisés.
- Développer un tableau de bord unifié (Grafana) consolidant les métriques de sécurité et de performance.

12.6.3 Long terme (> 12 mois)

- Évaluer le Post-Quantum TLS (ML-KEM / ML-DSA) pour anticiper la menace quantique sur la PKI [14].
- Explorer la Federated Learning pour entraîner les modèles de détection directement sur les passerelles IoT, sans centraliser les données.
- Proposer un framework open source packagé « IoT-Sec-Framework » réutilisable par d’autres organisations.
- Étendre l’approche à d’autres protocoles IoT : CoAP/DTLS, AMQP, LwM2M.

Ce projet a démontré qu’il est possible de sécuriser un écosystème IoT de bout en bout avec des outils open source, tout en maintenant des performances compatibles avec les contraintes du domaine. La combinaison PKI automatisée + TLS 1.3/mTLS + IDS + SIEM + ML offre une défense en profondeur solide et extensible.

Références bibliographiques

- [1] OASIS, *MQTT Version 5.0*, OASIS, 2019. adresse : <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [2] ENISA, *ENISA Threat Landscape for IoT*, Consulté en janvier 2024, 2023. adresse : <https://www.enisa.europa.eu/publications/enisa-threat-landscape-for-internet-of-things>
- [3] C. KOLIAS, G. KAMBOURAKIS, A. STAVROU et J. VOAS, « DDoS in the IoT : Mirai and Other Botnets, » *Computer*, t. 50, n° 7, p. 80-84, 2017. DOI : [10.1109/MC.2017.201](https://doi.org/10.1109/MC.2017.201)
- [4] E. RESCORLA, « The Transport Layer Security (TLS) Protocol Version 1.3, » Internet Engineering Task Force, Request for Comments RFC 8446, 2018. adresse : <https://www.rfc-editor.org/rfc/rfc8446>
- [5] NIST, « IoT Device Cybersecurity Guidance for the Federal Government, » National Institute of Standards et Technology, rapp. tech. SP 800-213, 2021. DOI : [10.6028/NIST.SP.800-213](https://doi.org/10.6028/NIST.SP.800-213)
- [6] HASHICORP, *Vault PKI Secrets Engine — Documentation*, Consulté en janvier 2024, 2024. adresse : <https://developer.hashicorp.com/vault/docs/secrets/pki>
- [7] OISF, *Suricata — MQTT Protocol Support*, Consulté en janvier 2024, 2023. adresse : <https://docs.suricata.io/en/latest/rules/mqtt-keywords.html>
- [8] WAZUH INC., *Wazuh Documentation — Integration with Suricata*, Consulté en janvier 2024, 2024. adresse : <https://documentation.wazuh.com/current/proof-of-concept-guide/detect-network-attacks-suricata.html>
- [9] F. T. LIU, K. M. TING et Z.-H. ZHOU, « Isolation Forest, » p. 413-422, 2008. DOI : [10.1109/ICDM.2008.17](https://doi.org/10.1109/ICDM.2008.17)
- [10] L. BREIMAN, « Random Forests, » *Machine Learning*, t. 45, n° 1, p. 5-32, 2001. DOI : [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
- [11] GNS3 TECHNOLOGIES, *GNS3 Documentation*, Consulté en novembre 2023, 2024. adresse : <https://docs.gns3.com>

- [12] ECLIPSE FOUNDATION, *Eclipse Mosquitto — TLS/SSL Configuration*, Consulté en décembre 2023, 2023. adresse : <https://mosquitto.org/man/mosquitto-conf-5.html>
- [13] F. PEDREGOSA et al., « Scikit-learn : Machine Learning in Python, » t. 12, 2011, p. 2825-2830.
- [14] Y. SHEFFER, P. SAINT-ANDRE et T. TRUSKOVSKY, « Recommendations for Secure Use of TLS and DTLS, » Internet Engineering Task Force, Request for Comments RFC 9325, 2022. adresse : <https://www.rfc-editor.org/rfc/rfc9325>

Annexe A

Scripts et Code Source

Cette annexe regroupe l'ensemble des scripts développés pour le projet. Ils sont classés par fonction.

A.1 Bootstrap de la PKI — Vault API

Ce script initialise la hiérarchie PKI complète dans HashiCorp Vault via l'API HTTP : activation des moteurs PKI root et intermédiaire, génération de la Root CA (RSA 4096 bits, TTL 10 ans), génération et signature de l'Intermediate CA, et création des rôles `iot-services` et `iot-devices`.

```
1  #!/bin/sh
2  set -e
3
4  VAULT="http://192.168.30.10:8200"
5  ROOT_TOKEN_FILE="/work/vault/root_token.txt"
6
7  if [ ! -f "$ROOT_TOKEN_FILE" ]; then
8      echo "root_token.txt introuvable. Fais d'abord init/unseal."
9      exit 1
10 fi
11
12 ROOT_TOKEN=$(cut -d= -f2 "$ROOT_TOKEN_FILE")
13
14 h() { echo "==> $1"; }
15 req() {
16     method=$1; path=$2; data=$3
17     if [ -n "$data" ]; then
18         curl -sS -X "$method" \
19             -H "X-Vault-Token: $ROOT_TOKEN" \
```

```

20     -H "Content-Type: application/json" \
21     --data "$data" \
22     "$VAULT$path"
23 else
24     curl -sS -X "$method" \
25     -H "X-Vault-Token: $ROOT_TOKEN" \
26     "$VAULT$path"
27 fi
28 }
29
30 # 1) Enable PKI engines
31 h "Enable PKI root at /pki"
32 req POST "/v1/sys/mounts/pki" \
33     '{"type":"pki","config":{"max_lease_ttl":"87600h"}}' || true
34
35 h "Enable PKI intermediate at /pki_int"
36 req POST "/v1/sys/mounts/pki_int" \
37     '{"type":"pki","config":{"max_lease_ttl":"43800h"}}' || true
38
39 # 2) Generate Root CA (internal)
40 h "Generate Root CA"
41 req POST "/v1/pki/root/generate/internal" '{
42     "common_name":"PFE-IoT-Security Root CA",
43     "organization":"FST / Tunisie Telecom",
44     "country":"TN",
45     "ttl":"87600h",
46     "key_type":"rsa",
47     "key_bits":4096
48 }' | tee /work/vault/root-ca.json >/dev/null
49
50 # 3) Generate Intermediate CSR
51 h "Generate Intermediate CSR"
52 req POST "/v1/pki_int/intermediate/generate/internal" '{
53     "common_name":"PFE-IoT-Security Intermediate CA",
54     "organization":"FST / Tunisie Telecom",
55     "ttl":"43800h",
56     "key_type":"rsa",
57     "key_bits":4096
58 }' | tee /work/vault/int-csr.json >/dev/null
59
60 CSR=$(jq -r '.data.csr' /work/vault/int-csr.json)

```

```

61
62 # 4) Sign Intermediate with Root
63 h "Sign Intermediate CSR with Root"
64 req POST "/v1/pki/root/sign-intermediate" \
65     "$(jq -n --arg csr "$CSR" '{
66         csr:$csr,
67         common_name:"PFE-IoT-Security Intermediate CA",
68         ttl:"43800h"
69     }')" | tee /work/vault/int-signed.json >/dev/null
70
71 CERT=$(jq -r '.data.certificate' /work/vault/int-signed.json)
72
73 # 5) Set signed Intermediate cert
74 h "Set signed Intermediate cert"
75 req POST "/v1/pki_int/intermediate/set-signed" \
76     "$(jq -n --arg cert "$CERT" '{certificate:$cert}')"
77     >/dev/null
78
79 # 6) Create roles
80 h "Create role iot-services"
81 req POST "/v1/pki_int/roles/iot-services" '{
82     "allowed_domains":"iot-pfe.local,dmz.iot-pfe.local",
83     "allow_subdomains":true,
84     "allow_bare_domains":true,
85     "max_ttl":"8760h",
86     "ttl":"720h",
87     "key_type":"rsa",
88     "key_bits":2048,
89     "server_flag":true,
90     "client_flag":true
91 }' >/dev/null
92
93 h "Create role iot-devices"
94 req POST "/v1/pki_int/roles/iot-devices" '{
95     "allowed_domains":"iot-pfe.local,iot.iot-pfe.local",
96     "allow_subdomains":true,
97     "max_ttl":"720h",
98     "ttl":"24h",
99     "key_type":"rsa",
100    "key_bits":2048,
101    "server_flag":false,

```

```

101     "client_flag":true
102 },' >/dev/null
103
104 echo "PKI bootstrap done."
```

Listing A.1 – bootstrap-pki-api.sh — Initialisation PKI via Vault API

A.2 Émission du certificat Mosquitto

```

1  #!/bin/sh
2  set -e
3  VAULT="http://192.168.30.10:8200"
4  ROOT_TOKEN=$(cut -d= -f2 /work/vault/root_token.txt)
5
6  mkdir -p /work/vault/certs
7
8  curl -sS -X POST \
9      -H "X-Vault-Token: $ROOT_TOKEN" \
10     -H "Content-Type: application/json" \
11     --data '{
12         "common_name":"mosquitto.dmz.iot-pfe.local",
13         "alt_names":"mqtt.iot-pfe.local,localhost",
14         "ip_sans":"192.168.20.10,127.0.0.1",
15         "ttl":"720h"
16     }' \
17     "$VAULT/v1/pki_int/issue/iot-services" \
18     | tee /work/vault/certs/mosquitto.json | jq .
19
20 jq -r '.data.certificate' \
21     /work/vault/certs/mosquitto.json >
22     /work/vault/certs/mosquitto.crt
23 jq -r '.data.private_key' \
24     /work/vault/certs/mosquitto.json >
25     /work/vault/certs/mosquitto.key
26
27 jq -r '.data.issuing_ca' \
28     /work/vault/certs/mosquitto.json >
29     /work/vault/certs/intermediate-ca.crt
30
31 echo "Certificat Mosquitto emis avec succes."
```

Listing A.2 – issue-mosquitto-cert-api.sh — Émission du certificat du broker

A.3 Génération des certificats devices

Ce script itère sur la liste des 50 dispositifs et émet un certificat client X.509 pour chacun via le rôle `iot-devices` de Vault.

```

1  #!/bin/sh
2  set -e
3
4  VAULT="http://192.168.30.10:8200"
5  ROOT_TOKEN="$(cut -d= -f2 /work/vault/root_token.txt)"
6  LIST="/work/fleet-sim/out/devices_top50.txt"
7  OUT="/work/fleet-sim/certs"
8
9  if [ ! -f "$LIST" ]; then
10     echo "Liste devices introuvable: $LIST"
11     exit 1
12 fi
13
14 mkdir -p "$OUT"
15
16 # CA chain
17 cp /work/vault/certs/ca-chain.crt "$OUT/ca-chain.crt"
18
19 echo "==> Generating device certs"
20
21 i=0
22 while IFS= read -r DEV; do
23     DEV=$(echo "$DEV" | tr -d '\r')
24     [ -z "$DEV" ] && continue
25     i=$((i+1))
26     echo "[$i] $DEV"
27
28     DEV_DNS=$(echo "$DEV" | tr '_' '-')
29     mkdir -p "$OUT/$DEV"
30
31     curl -sS -X POST \
32         -H "X-Vault-Token: $ROOT_TOKEN" \
33         -H "Content-Type: application/json" \
34         --data
35         "{ \"common_name\": \"$DEV_DNS.iot.iot-pfe.local\", \"ttl\": \"720h\" }"
36         \
37         "$VAULT/v1/pki_int/issue/iot-devices" \

```

```

36     > "$OUT/$DEV/issue.json"
37
38     jq -r '.data.certificate' "$OUT/$DEV/issue.json" >
        "$OUT/$DEV/client.crt"
39     jq -r '.data.private_key' "$OUT/$DEV/issue.json" >
        "$OUT/$DEV/client.key"
40 done < "$LIST"
41
42 echo "Done. Generated $i device certs."

```

Listing A.3 – generate_device_certs.sh — Certificats pour 50 devices IoT

A.4 Génération du fichier ACL Mosquitto

```

1  #!/bin/sh
2  set -e
3
4  LIST="/work/fleet-sim/out/devices_top50.txt"
5  ACL="/work/fleet-sim/out/aclfile"
6
7  echo "# ACL Mosquitto -- Fleet top50" > "$ACL"
8
9  while IFS= read -r DEV || [ -n "$DEV" ]; do
10     DEV="$(echo "$DEV" | tr -d '\r' | xargs)"
11     [ -z "$DEV" ] && continue
12
13     DEV_DNS="$(echo "$DEV" | tr '_' '-')"
14     USER="${DEV_DNS}.iot.iot-pfe.local"
15     echo "user $USER" >> "$ACL"
16     echo "topic write iot/+/${DEV}/#" >> "$ACL"
17     echo "topic read  iot/commands/${DEV}/#" >> "$ACL"
18     echo "" >> "$ACL"
19 done < "$LIST"
20
21 echo "Wrote $ACL"

```

Listing A.4 – generate_aclfile.sh — Création des ACL par device

A.5 Tests de connectivité réseau

```

1  #!/bin/bash
2  # PFE IoT Security TT -- Test de Connectivite Reseau
3  pass=0; fail=0
4
5  test_ping() {
6      local from=$1; local to_ip=$2; local desc=$3; local
          expected=$4
7      result=$(docker exec $from ping -c 1 -W 2 $to_ip 2>&1)
8      if echo "$result" | grep -q "1 received"; then
9          if [ "$expected" = "pass" ]; then
10             echo "PASS -- $desc ($from -> $to_ip)"; ((pass++))
11         else
12             echo "FAIL -- $desc ($from -> $to_ip) -- DEVRAIT ETRE
                  BLOQUE"; ((fail++))
13         fi
14     else
15         if [ "$expected" = "fail" ]; then
16             echo "BLOQUE (attendu) -- $desc ($from -> $to_ip)";
                  ((pass++))
17         else
18             echo "FAIL -- $desc ($from -> $to_ip) -- PAS DE
                  REPONSE"; ((fail++))
19         fi
20     fi
21 }
22
23 # Tests de connectivite autorisee
24 test_ping "test-iot" "192.168.10.1" "IoT -> Gateway
          MikroTik" "pass"
25 test_ping "test-dmz" "192.168.20.1" "DMZ -> Gateway
          MikroTik" "pass"
26 test_ping "test-pki" "192.168.30.1" "PKI -> Gateway
          MikroTik" "pass"
27
28 # Tests de segmentation (doit etre bloque)
29 test_ping "test-iot" "192.168.40.10" "IoT -> SIEM (BLOQUE)"
          "fail"
30 test_ping "test-iot" "192.168.50.10" "IoT -> Analyse
          (BLOQUE)" "fail"
31
32 echo "Resultats : $pass PASS / $fail FAIL"

```

Listing A.5 – test-connectivity.sh — Validation de la segmentation réseau

A.6 Tests de performance MQTT

Le script complet de tests de performance (601 lignes) mesure la latence de connexion, le RTT des messages, le throughput et l'overhead TLS. Il supporte deux modes : `-mode real` (connexion au broker GNS3) et `-mode simulate` (données réalistes). Seule la structure principale est reproduite ci-dessous ; le code complet est disponible dans le dépôt (`tests/performance/mqtt_perf_test.py`).

```

1 Phase 9 -- Tests de performance MQTT
2 PFE -- Sécurisation des flux de communication IoT
3 FST / Tunisie Telecom | Amine Ghannouchi
4
5 import argparse, json, os, ssl, time, threading, statistics
6 import numpy as np, matplotlib.pyplot as plt
7
8 BROKER_HOST      = '192.168.20.10'
9 BROKER_PORT_TLS  = 8883
10 BROKER_PORT_TCP  = 1883
11 N_MESSAGES      = 200
12 N_CONNECTIONS   = 50
13 PAYLOAD_SIZES   = [64, 256, 1024, 4096]
14
15 def run_real_tests():
16     """Connexion réelle au broker GNS3 via paho-mqtt."""
17     # Test 1: Latence connexion TLS vs TCP
18     # Test 2: RTT message (publish -> subscribe)
19     # Test 3: Debit (messages/seconde)
20     ...
21
22 def run_simulation():
23     """Donnees realistes basees sur RFC 8446 et benchmarks."""
24     rng = np.random.default_rng(42)
25     conn_tcp = rng.normal(loc=8.5, scale=2.1,
26                           size=N_CONNECTIONS)
27     conn_tls = rng.normal(loc=42.3, scale=7.8,
28                           size=N_CONNECTIONS)
29     rtt_tcp  = rng.normal(loc=4.2, scale=1.1,
30                           size=N_MESSAGES)

```

```

28     rtt_tls = rng.normal(loc=7.8, scale=1.9,
29                             size=N_MESSAGES)
30     ...
31 def generate_plots(data):
32     """5 graphiques: connexion, RTT, throughput, protocoles,
33         handshake."""
34     ...
35 def print_summary(data):
36     """Resume textuel + export JSON."""
37     ...
38
39 if __name__ == '__main__':
40     parser = argparse.ArgumentParser()
41     parser.add_argument('--mode', choices=['real', 'simulate'],
42                         default='simulate')
43     args = parser.parse_args()
44     data = run_real_tests() if args.mode == 'real' else
45         run_simulation()
46     generate_plots(data)
47     print_summary(data)

```

Listing A.6 – mqtt_perf_test.py — Structure du script de performance

A.7 Simulateur de flotte IoT

Le simulateur (`fleet_sim.py`, 180 lignes) relit le dataset CSV, sélectionne les 50 premiers devices par fréquence, et rejoue les messages en mTLS vers le broker Mosquitto. Chaque device se connecte avec son propre certificat client.

```

1 import argparse, json, os, ssl, time, pandas as pd
2 import paho.mqtt.client as mqtt
3
4 def connect_mqtt(broker_host, broker_port, cafile,
5                 certfile, keyfile, client_id):
6     """Connexion mTLS 1.3 au broker MQTT."""
7     client = mqtt.Client(client_id=client_id)
8     ctx = ssl.create_default_context(cafile=cafile)
9     ctx.load_cert_chain(certfile=certfile, keyfile=keyfile)
10    ctx.minimum_version = ssl.TLSVersion.TLSv1_3

```

```

11     client.tls_set_context(ctx)
12     client.connect(broker_host, broker_port, keepalive=60)
13     client.loop_start()
14     return client
15
16 def replay_device(df_dev, args, device_id):
17     """Rejoue les messages d'un device en respectant le
18         timing."""
19     client = connect_mqtt(...)
20     for _, row in df_dev.iterrows():
21         topic =
22             f"iot/{row['tenant_id']}/{row['device_id']}/telemetry"
23         payload = json.dumps({...})
24         # Comportements d'attaque: DoS burst, replay
25         if row["attack_type"] == "dos":
26             for _ in range(args.dos_burst):
27                 client.publish(topic, payload, qos=1)
28         else:
29             client.publish(topic, payload, qos=1)
30     client.disconnect()
31
32 def main():
33     df = pd.read_csv(args.csv)
34     chosen = df["device_id"].value_counts().head(50).index
35     for device_id in chosen:
36         replay_device(df[df["device_id"] == device_id], args,
37                       device_id)
38
39 if __name__ == "__main__":
40     main()

```

Listing A.7 – fleet_sim.py — Simulateur de flotte IoT (structure)

A.8 Analyse des événements Suricata

```

1 #!/usr/bin/env python3
2 """Analyse du eve.json Suricata - Phase 6"""
3 import json, pandas as pd, matplotlib.pyplot as plt
4 from pathlib import Path
5 from collections import Counter
6

```

```

7 EVE = Path("results/analysis/step6/suricata-v2/eve.json")
8
9 events = []
10 with open(EVE) as f:
11     for line in f:
12         if line.strip():
13             events.append(json.loads(line))
14
15 df = pd.DataFrame(events)
16 df["timestamp"] = pd.to_datetime(df["timestamp"], utc=True)
17
18 print(f"Total events : {len(df)}")
19 print(f"Event types :
20       {df['event_type'].value_counts().to_dict()}")
21
22 # Connexions TLS
23 tls = df[df["event_type"] == "tls"]
24 print(f"TLS connections : {len(tls)}")
25
26 # Alertes Suricata
27 alerts = df[df["event_type"] == "alert"]
28 print(f"Alerts : {len(alerts)}")
29 if not alerts.empty:
30     sigs = alerts["alert"].apply(lambda x: x.get("signature",
31         "?"))
32     print(sigs.value_counts().to_string())
33
34 # Timeline des connexions TLS
35 if not tls.empty:
36     tls = tls.set_index("timestamp").sort_index()
37     fig, ax = plt.subplots(figsize=(12, 4))
38     tls.resample("5s")["src_ip"].count().plot(ax=ax,
39         color="steelblue")
40     ax.set_title("Connexions TLS/MQTT par tranche de 5
41         secondes")
42     plt.savefig("results/analysis/step6/tls_connections_timeline.png")

```

Listing A.8 – analyze_eve.py — Analyse du fichier eve.json Suricata

Annexe B

Fichiers de Configuration

Cette annexe regroupe les fichiers de configuration des différents composants de l'infrastructure.

B.1 Configuration MikroTik CHR

Configuration complète du routeur MikroTik CHR : bridges par zone (VLAN), adresses IP des passerelles, règles de firewall inter-VLANs, et port mirroring vers Suricata.

```
1  # PFE IoT Security TT -- MikroTik CHR Configuration
2
3  /system identity set name=mikrotik-pfe
4
5  # Creer les bridges par VLAN
6  /interface bridge
7  add name=bridge-iot      comment="Zone IoT - VLAN 10"
8  add name=bridge-dmz      comment="Zone DMZ - VLAN 20"
9  add name=bridge-pki      comment="Zone PKI - VLAN 30"
10 add name=bridge-siem     comment="Zone SIEM - VLAN 40"
11 add name=bridge-analyse  comment="Zone Analyse - VLAN 50"
12
13 # Assigner les ports aux bridges
14 /interface bridge port
15 add bridge=bridge-iot     interface=ether2
16 add bridge=bridge-dmz    interface=ether3
17 add bridge=bridge-pki    interface=ether4
18 add bridge=bridge-siem   interface=ether5
19 add bridge=bridge-analyse interface=ether6
20
```

```
21 # Adresses IP des passerelles (routeur L3)
22 /ip address
23 add address=192.168.1.2/24 interface=ether1
   comment="Uplink pfSense"
24 add address=192.168.10.1/24 interface=bridge-iot
   comment="GW Zone IoT"
25 add address=192.168.20.1/24 interface=bridge-dmz
   comment="GW Zone DMZ"
26 add address=192.168.30.1/24 interface=bridge-pki
   comment="GW Zone PKI"
27 add address=192.168.40.1/24 interface=bridge-siem
   comment="GW Zone SIEM"
28 add address=192.168.50.1/24 interface=bridge-analyse
   comment="GW Zone Analyse"
29
30 # Route par défaut vers pfSense
31 /ip route
32 add dst-address=0.0.0.0/0 gateway=192.168.1.1
33
34 # Firewall -- ACL inter-VLANs
35 /ip firewall filter
36 add chain=forward connection-state=established,related
   action=accept
37 add chain=forward src-address=192.168.10.0/24
   dst-address=192.168.20.10 \
38   protocol=tcp dst-port=8883 action=accept
   comment="IoT->MQTT TLS"
39 add chain=forward src-address=192.168.10.0/24
   dst-address=192.168.30.10 \
40   protocol=tcp dst-port=8200 action=accept
   comment="IoT->Vault PKI"
41 add chain=forward src-address=192.168.20.0/24
   dst-address=192.168.30.10 \
42   protocol=tcp dst-port=8200 action=accept
   comment="DMZ->Vault verify"
43 add chain=forward src-address=192.168.20.0/24
   dst-address=192.168.40.10 \
44   protocol=tcp dst-port=1514 action=accept
   comment="DMZ->Wazuh logs"
45 add chain=forward src-address=192.168.50.0/24
   dst-address=192.168.40.10 \
```

```

46     protocol=tcp dst-port=1514 action=accept
         comment="Analyse->Wazuh"
47 add chain=forward src-address=192.168.10.0/24
         dst-address=192.168.40.0/24 \
48     action=drop comment="BLOCK IoT->SIEM direct"
49 add chain=forward src-address=192.168.10.0/24
         dst-address=192.168.50.0/24 \
50     action=drop comment="BLOCK IoT->Analyse direct"
51 add chain=forward action=drop comment="DROP all other
         inter-VLAN"

```

Listing B.1 – mikrotik-config.rsc — Configuration du routeur MikroTik

B.2 Configuration Mosquitto (TLS/mTLS)

```

1  # =====
2  # Mosquitto v2.0 -- Configuration TLS 1.3 / mTLS
3  # PFE IoT Security TT
4  # =====
5
6  # Listener TLS (port securise)
7  listener 8883
8
9  # Certificats serveur
10 cafile      /mosquitto/certs/ca-chain.crt
11 certfile    /mosquitto/certs/mosquitto.crt
12 keyfile     /mosquitto/certs/mosquitto.key
13
14 # mTLS : exiger certificat client
15 require_certificate true
16 use_identity_as_username true
17
18 # Version TLS minimale
19 tls_version tlsv1.3
20
21 # Ciphersuites TLS 1.3
22 ciphers_tls1.3
         TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
23
24 # ACL basee sur le CN du certificat
25 acl_file /mosquitto/config/aclfile

```

```
26
27 # Logging
28 log_dest stdout
29 log_type all
30 connection_messages true
31
32 # Securite
33 allow_anonymous false
34 max_connections 200
35 max_inflight_messages 20
36 max_queued_messages 1000
37
38 # Persistence
39 persistence true
40 persistence_location /mosquitto/data/
```

Listing B.2 – mosquitto.conf — Broker MQTT avec TLS 1.3 et mTLS

B.3 Configuration Suricata (MQTT natif)

Extrait de la configuration Suricata v7.0 pertinente pour la détection MQTT.

```
1 # Suricata v7.0 -- Configuration pour detection MQTT
2 # PFE IoT Security TT
3
4 vars:
5   address-groups:
6     IOT_NET:      "192.168.10.0/24"
7     DMZ_NET:      "192.168.20.0/24"
8     MQTT_BROKER: "192.168.20.10"
9
10  app-layer:
11    protocols:
12      mqtt:
13        enabled: yes
14        ports: [1883, 8883]
15      tls:
16        enabled: yes
17        ports: [8883, 443]
18
19  outputs:
20    - eve-log:
```

```
21     enabled: yes
22     filetype: regular
23     filename: eve.json
24     types:
25         - alert:
26             tagged-packets: yes
27             metadata: yes
28         - mqtt:
29             passwords: no
30         - tls:
31             extended: yes
32         - stats:
33             totals: yes
34             threads: no
35
36 rule-files:
37     - suricata.rules
38     - mqtt-custom.rules
```

Listing B.3 – suricata.yaml — Extrait de la configuration IDS

B.4 Règles Suricata personnalisées

```
1  # =====
2  # Regles Suricata personnalisees -- Detection MQTT IoT
3  # PFE IoT Security TT
4  # =====
5
6  # SID 1000001 -- Connexion MQTT sans TLS (port 1883)
7  alert mqtt any any -> $MQTT_BROKER 1883 (msg:"PFE - MQTT sans
8      TLS detecte"; \
9      flow:to_server,established; sid:1000001; rev:1; \
10     classtype:policy-violation;)
11
12 # SID 1000002 -- Flood MQTT (>50 PUBLISH en 10s)
13 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
14     flood detecte"; \
15     flow:to_server,established; mqtt.type:PUBLISH; \
16     threshold:type both, track by_src, count 50, seconds 10; \
17     sid:1000002; rev:1; classtype:attempted-dos;)
```

```

17 # SID 1000003 -- Subscribe wildcard '#'
18 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
    subscribe wildcard #"; \
19     flow:to_server,established; mqtt.type:SUBSCRIBE; \
20     content:"#"; sid:1000003; rev:1; classtype:policy-violation;)
21
22 # SID 1000004 -- Payload anormalement grand (>4096 octets)
23 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
    payload > 4096B"; \
24     flow:to_server,established; mqtt.type:PUBLISH; \
25     dsize:>4096; sid:1000004; rev:1; classtype:bad-unknown;)
26
27 # SID 1000005 -- Tentative connexion depuis zone non-IoT
28 alert mqtt !$IOT_NET any -> $MQTT_BROKER 8883 (msg:"PFE - MQTT
    depuis zone non-IoT"; \
29     flow:to_server; sid:1000005; rev:1;
    classtype:policy-violation;)

```

Listing B.4 – mqtt-custom.rules — Règles IDS personnalisées MQTT

B.5 Règles Wazuh personnalisées

```

1 <!-- Wazuh custom rules -- PFE IoT Security TT -->
2 <group name="pfe-iot,suricata,mqtt">
3
4     <!-- Alerte Suricata MQTT generique -->
5     <rule id="100100" level="5">
6         <if_sid>86601</if_sid>
7         <field name="alert.signature_id">~1000</field>
8         <description>PFE -- Alerte Suricata MQTT
            detectee</description>
9         <group>pfe-iot,mqtt,suricata</group>
10    </rule>
11
12    <!-- MQTT flood (DoS) -->
13    <rule id="100101" level="10">
14        <if_sid>100100</if_sid>
15        <field name="alert.signature_id">1000002</field>
16        <description>PFE -- MQTT DoS flood detecte (>50
            msg/10s)</description>
17        <group>pfe-iot,mqtt,dos</group>

```

```

18     <options>alert_by_email</options>
19 </rule>
20
21 <!-- Subscribe wildcard -->
22 <rule id="100102" level="8">
23     <if_sid>100100</if_sid>
24     <field name="alert.signature_id">1000003</field>
25     <description>PFE -- MQTT subscribe wildcard #
        detecte</description>
26     <group>pfe-iot,mqtt,policy-violation</group>
27 </rule>
28
29 <!-- Connexion MQTT sans TLS -->
30 <rule id="100103" level="12">
31     <if_sid>100100</if_sid>
32     <field name="alert.signature_id">1000001</field>
33     <description>PFE -- MQTT sans TLS (violation de
        politique)</description>
34     <group>pfe-iot,mqtt,tls-violation</group>
35     <options>alert_by_email</options>
36 </rule>
37
38 <!-- Correlation : 3+ alertes MQTT du meme device en 60s -->
39 <rule id="100110" level="13" frequency="3" timeframe="60">
40     <if_matched_sid>100100</if_matched_sid>
41     <same_source_ip />
42     <description>PFE -- Activite suspecte repetee depuis le
        meme device IoT</description>
43     <group>pfe-iot,mqtt,correlation</group>
44 </rule>
45
46 </group>

```

Listing B.5 – local_rules.xml — Règles Wazuh pour corrélation MQTT/Suricata

B.6 Dockerfile du simulateur de flotte

```

1 FROM python:3.11-alpine
2
3 RUN apk add --no-cache ca-certificates openssl \
4     && update-ca-certificates

```

```

5
6 WORKDIR /app
7 COPY app/requirements.txt /app/requirements.txt
8 RUN pip install --no-cache-dir -r /app/requirements.txt
9
10 COPY app/fleet_sim.py /app/fleet_sim.py
11 COPY certs/ /app/certs/
12 COPY dataset/ /app/dataset/
13
14 CMD ["python", "/app/fleet_sim.py", \
15     "--csv",
16     "/app/dataset/iot_communication_security_dataset_sample.csv"]

```

Listing B.6 – Dockerfile — Image Docker du simulateur de flotte IoT

B.7 Dockerfile Toolbox

```

1 FROM alpine:3.20
2
3 RUN apk add --no-cache \
4     bash ca-certificates curl openssl \
5     bind-tools iproute2 iputils tcpdump \
6     net-tools nmap jq \
7     python3 py3-pip git \
8     busybox-extras tzdata \
9     && update-ca-certificates
10
11 RUN adduser -D pfe && mkdir -p /work && chown -R pfe:pfe /work
12 WORKDIR /work
13 USER pfe
14
15 CMD ["bash"]

```

Listing B.7 – Dockerfile — Image toolbox pour tests réseau et sécurité

B.8 Commandes de création des réseaux Docker

```

1 # VLAN 10 -- Zone IoT
2 docker network create --driver bridge \
3     --subnet 192.168.10.0/24 --gateway 192.168.10.1 \

```

```
4  --opt com.docker.network.bridge.name=br-iot pfe-iot-network
5
6  # VLAN 20 -- Zone DMZ
7  docker network create --driver bridge \
8  --subnet 192.168.20.0/24 --gateway 192.168.20.1 \
9  --opt com.docker.network.bridge.name=br-dmz pfe-dmz-network
10
11 # VLAN 30 -- Zone PKI
12 docker network create --driver bridge \
13 --subnet 192.168.30.0/24 --gateway 192.168.30.1 \
14 --opt com.docker.network.bridge.name=br-pki pfe-pki-network
15
16 # VLAN 40 -- Zone SIEM
17 docker network create --driver bridge \
18 --subnet 192.168.40.0/24 --gateway 192.168.40.1 \
19 --opt com.docker.network.bridge.name=br-siem pfe-siem-network
20
21 # VLAN 50 -- Zone Analyse
22 docker network create --driver bridge \
23 --subnet 192.168.50.0/24 --gateway 192.168.50.1 \
24 --opt com.docker.network.bridge.name=br-analyse
    pfe-analyse-network
```

Listing B.8 – Création des réseaux Docker par zone de sécurité

B.9 Variables d'environnement du projet

TABLE B.1 – Variables d'environnement principales du projet

Variable	Valeur	Description
VAULT_ADDR	http://192.168.30.10:8200	Adresse de l'API Vault
VAULT_TOKEN	<root_token>	Token d'authentification
MQTT_BROKER	192.168.20.10	IP du broker Mosquitto
MQTT_PORT_TLS	8883	Port MQTT sécurisé
MQTT_PORT_TCP	1883	Port MQTT non sécurisé
CA_CERT	/certs/ca-chain.crt	Chaîne CA (root + intermediate)
WAZUH_MANAGER	192.168.40.10	IP du manager Wazuh
SURICATA_EVE	/var/log/suricata/eve.json	Fichier événements Suricata